

Smart City Energy Optimization & Prediction Platform



Team 4 Names:

- I. Merna Sameh Mahmoud
- II. Nagat Osama
- III. Ebrahim Ateff
- IV. Abdullah Mohamed

Under the supervision of:
Eng Ahmed Ali

Table of Contents

Executive Summary:	2
1. Introduction	3
1.1 Problem Statement	3
1.2 Solution Overview	3
1.3 Project Goals	4
System Architecture	5
2.1 Architecture Diagram	5
2.2 Components Used	6
2.3 Medallion Architecture	6
Data Engineering Pipelines	9
Pipeline 1 — Ingest Energy (CSV → ADLS Raw → Bronze)	9
Pipeline 2 — Ingest Weather API (REST → Raw → Bronze)	13
Pipeline 3 — Ingest Traffic API	16
ADF Monitoring & Alerts	19
Monitoring	19
Alerts (Azure Monitor Integration)	20

Executive Summary:

Azure Data Engineering and Machine Learning services. The system integrates batch energy data, real-time weather API, and traffic API to build a unified city-level intelligence model. After processing and enriching the data through the Medallion Architecture (Raw → Bronze → Silver → Gold), the platform predicts hourly and daily energy consumption, identifies peak load zones, and analyzes how weather and traffic variables influence consumption patterns.

The final solution includes an automated data ingestion pipeline (Azure Data Factory), a full cleaning and transformation layer (Databricks), a forecasting model built using Azure Machine Learning Designer, analytical views hosted in Synapse Serverless, and an interactive Power BI dashboard. The system delivers accurate forecasts, risk detection, and optimization recommendations to support stable and efficient energy distribution across the smart city.

1. Introduction

1.1 Problem Statement

Modern smart cities are experiencing rapid growth in population, infrastructure, and mobility. As a result, **energy consumption is increasing at an unpredictable rate**, causing pressure on the electric grid, especially during peak hours. Weather conditions such as temperature, humidity, and cloud cover directly influence household and industrial energy usage. Traffic congestion also plays a significant role, as longer travel times and rush-hour patterns increase the overall power demand across different regions.

Without an automated system capable of analyzing these variables and predicting future energy needs, cities face challenges such as:

- Sudden peak loads that can cause electrical instability
- Inefficient energy distribution across districts
- Difficulty planning for power shortages or high-demand periods
- Lack of insights to support decision-making for city planners

These issues highlight the need for an intelligent solution that can forecast consumption and identify risk zones before problems occur.

1.2 Solution Overview

This project provides a complete **Smart City Energy Optimization and Forecasting Platform** that integrates three essential data streams:

1. **Energy consumption data (batch CSV)**
2. **Weather data (external API)**

3. Traffic congestion data (external API)

The system processes these datasets through a modern **Medallion Architecture (Raw → Bronze → Silver → Gold)** using Azure cloud services.

The platform delivers:

- Clean, unified, and enriched data using Azure Databricks
- Accurate energy forecasting using Azure Machine Learning Designer
- Risk zone identification for regions with expected high loads
- Optimization recommendations that help balance the grid
- An interactive end-to-end dashboard built in Power BI

The solution aims to support smart city decision-makers with reliable analytics and real-time insights to improve energy stability and operational efficiency.

1.3 Project Goals

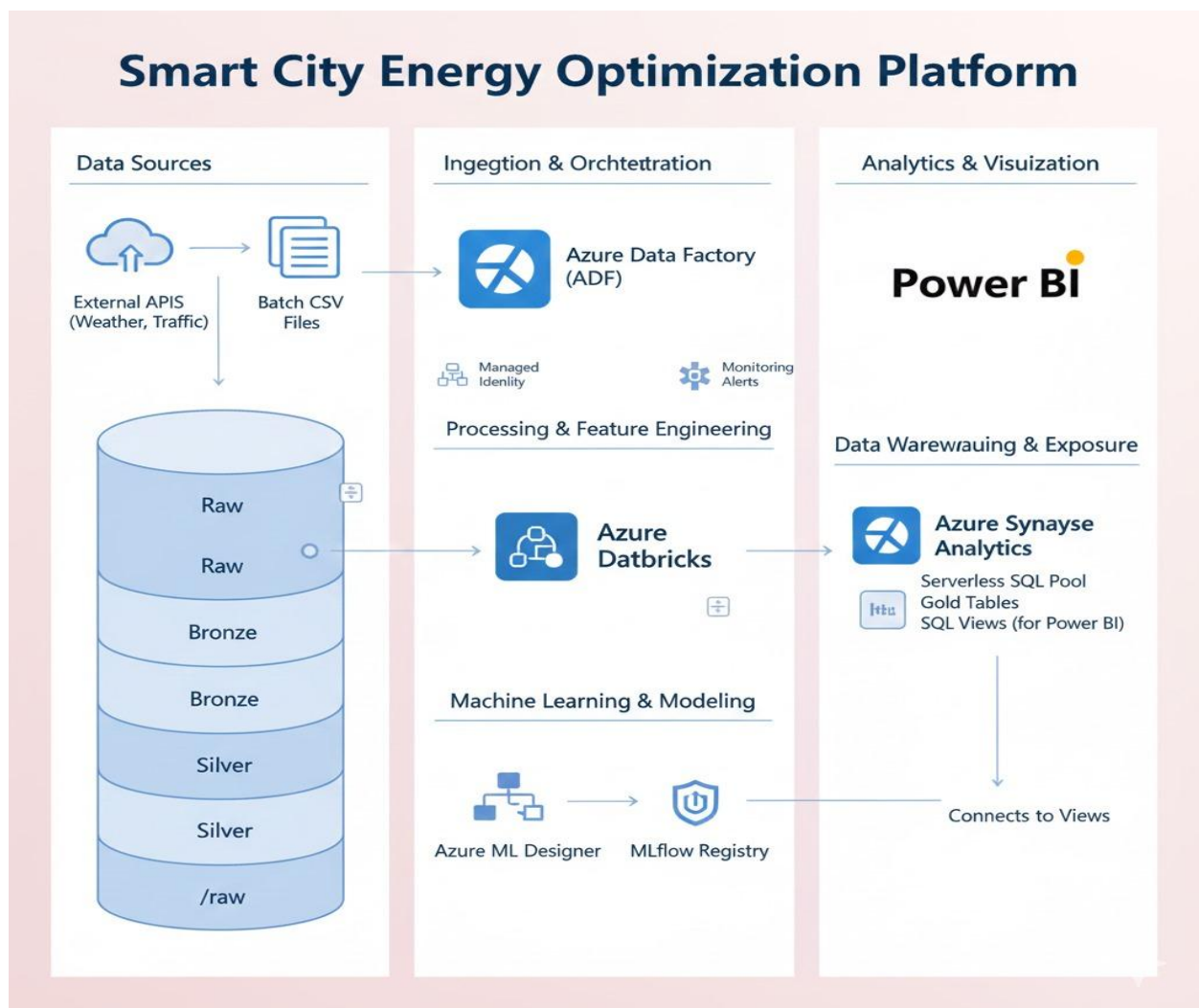
The main goals of this project are:

- **Predict hourly and daily energy consumption** for different city regions
- **Create risk zones** based on forecasted peak load levels
- **Develop an optimization engine** to recommend load shifting and resource balancing

- **Build a fully interactive Power BI dashboard** for data visualization and decision support
- **Implement an automated Azure pipeline** from ingestion to ML to reporting
- **Provide a scalable and reusable architecture** suitable for smart city applications

System Architecture

2.1 Architecture Diagram



2.2 Components Used

- Azure Data Lake
 - Azure Data Factory
 - Azure Databricks
 - Azure ML Designer
 - Synapse Serverless
 - Power BI
-

2.3 Medallion Architecture

The solution follows the **Medallion Architecture**, which is a modern data engineering framework designed to organize data into multiple incremental layers. Each layer adds more structure, quality, and business value to the data as it flows through the system. This architecture ensures scalability, reliability, and clarity for large data pipelines used in smart city environments.

Raw Layer — Landing Zone

This layer stores all incoming data **exactly as received**, without any processing or modification.

It serves as the single source of truth and provides full traceability.

In our project, Raw contains:

- Energy CSV files
- Weather API responses
- Traffic API responses

Characteristics:

- No transformations
 - No cleaning
 - Used for auditing and backup
-

Bronze Layer — Ingestion Layer

This layer contains data **after ingestion** using Azure Data Factory. The Bronze layer adds essential metadata and stores the data in a consistent, queryable format.

Transformations applied:

- Adding metadata columns (load_date, source_file_name, status)
- Normalizing API responses
- Converting CSV/API files into Delta-friendly structures

This layer ensures:

- Unified structure
 - Reliable ingestion
 - Separation between raw data and cleaned data
-

Silver Layer — Cleaning & Enrichment

The Silver Layer is where **the real data processing happens**. Using Databricks, data is cleaned, validated, enriched, and joined.

Transformations include:

- Removing duplicates

- Fixing timestamps and formats
- Handling missing values
- Feature engineering (weather_index, congestion_index, peak_hour_flag)
- Joining energy + weather + traffic into one unified master table

Output:

A high-quality analytics-ready table → **silver.smartcity_master**

Gold Layer — Business & ML Layer

The Gold Layer contains **the final analytical and machine learning outputs**.

This layer powers dashboards, reports, and optimization recommendations.

Gold layer contains:

1. **gold.energy_forecast**
→ Forecasted energy consumption using Azure ML Designer (ARIMA/Prophet)
2. **gold.risk_zones**
→ Classification of city regions into Low / Medium / High risk
3. **gold.optimization_recommendations**
→ Smart suggestions like load shifting and peak management

4. Synapse SQL views

→ Exposed directly to Power BI for business reporting

3. Data Engineering Pipelines → Bronze Layer

in this layer we created

1. Create ADLS folders: Raw / Bronze / Silver / Gold in storage account

2. Create Linked Services:

Linked Service for ADLS

Linked Service for APIs (Weather/Traffic)

3. Pipeline 1 — Ingest Energy (CSV → ADLS Raw → Bronze)

Objective

Ingest daily CSV files containing energy consumption data, enrich them with metadata, and store the cleaned output in the Bronze zone for downstream processing.

Pipeline Structure

The screenshot displays the Microsoft Azure Data Factory (ADF) interface for a pipeline named 'Ingest_Energy'. The left sidebar shows the 'Factory Resources' tree with 'Pipelines' expanded, listing 'Ingest_Energy' and its sub-pipelines: 'Ingest_Traffic_API', 'Ingest_Weather_API', and 'Change Data Capture (preview)'. The main canvas shows the pipeline's visual representation, starting with a 'Get Metadata' activity followed by a 'Get_Files' activity, which then feeds into a 'ForEach' loop containing a 'Copy data1' activity. The bottom panel shows the configuration for the 'Copy data1' activity, including its name, description, activity state (set to 'Activated'), timeout (0.12:00:00), retry (0), and retry interval (30 seconds).

Copy Data source/sink configuration

GeneralSourceSinkMappingSettingsUser properties

Source dataset *

ds_raw_smartcity

OpenNewPreview dataLearn more

Dataset properties

NameValue

FileName_raw@item().name

File path type

File path in dataset

Wildcard file path

List of files

Filter by last modified

Start time (UTC)

End time (UTC)

Recursively

☒

Enable partitions discovery

☐

GeneralSourceSinkMappingSettingsUser properties

Filter by last modified

Recursively

☒

Enable partitions discovery

☐

Max concurrent connections

Skip line count

Additional columns

+ New

Delete

NameValue

ingestion_timestamp@utcnow()

source_file_name@item().name

GeneralSourceSinkMappingSettingsUser properties

Sink dataset *

ds_bronze_smartcity

OpenNewLearn more

Dataset properties

NameValue

FileName_bronze@item().name

Copy behavior

Select...

Max concurrent connections

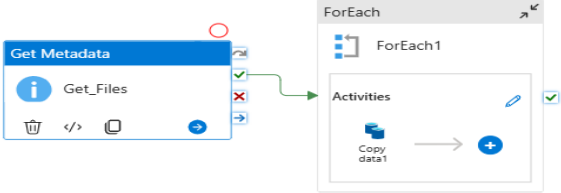
Block size (MB)

Metadata

+ New

Metadata settings

✓ Validate ▶ Debug ⚡ Trigger (1)



The diagram shows a workflow with two activities. The first activity is 'Get Metadata' (blue box) with a sub-activity 'Get_Files' (blue box). It is connected to a 'ForEach' loop (grey box) which contains a 'ForEach1' sub-loop (blue box) and an 'Activities' section (white box) with a 'Copy data1' activity (blue box). A green arrow connects the output of 'Get_Files' to the 'ForEach' loop.

General Settings User properties

Dataset * ds_raw_smartcity [Open](#) [+ New](#) [Learn more](#)

Dataset properties ⓘ

Name	Value
FileName_raw	*

Field list * [+ New](#) [Delete](#)

☐ Argument

☐ Child items

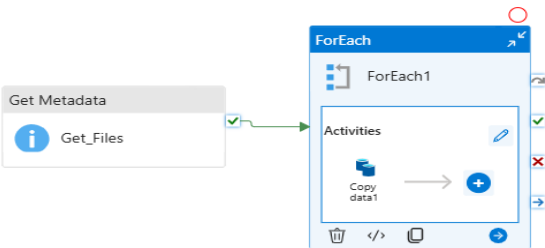
Filter by last modified ⓘ

Start time (UTC)

End time (UTC)

ForEach settings

✓ Validate ▶ Debug ⚡ Trigger (1)



The diagram shows a workflow with two activities. The first activity is 'Get Metadata' (grey box) with a sub-activity 'Get_Files' (blue box). It is connected to a 'ForEach' loop (blue box) which contains a 'ForEach1' sub-loop (blue box) and an 'Activities' section (white box) with a 'Copy data1' activity (blue box). A green arrow connects the output of 'Get_Files' to the 'ForEach' loop.

General Settings Activities (1) User properties

Sequential ☒

Items @activity('Get_Files').output.childItems

Trigger details

ift Fabric, Microsoft's newest cloud-first data analytics SaaS platform? Click [here](#) to get started with Fabric Data Factory!

Publish all

Activities

Get Metadata

Get_Files

Settings

Sequential

Items

@activity('Get_Files').output.childItems

Edit trigger

Name *

Daily_Files_Trigger

Description

Type *

ScheduleTrigger

Start date *

11/23/2025, 9:47:00 AM

Time zone *

Cairo (UTC+2)

Recurrence *

Every 1 Day(s)

Advanced recurrence options

Execute at these times

Hours 1

Minutes 0

Schedule execution times

01:00

Specify an end date

Annotations

+ New

Status

Started

OK Cancel

Successful pipeline run

Microsoft Azure | Data Factory | ADLSGen1111

Would you like to see Data Factory inside of Microsoft Fabric, Microsoft's newest cloud-first data analytics SaaS platform? Click [here](#) to get started with Fabric Data Factory!

Factory Resources

Pipelines

smart city

ingest_Energy

ingest_traffic_API

ingest_weather_API

Datasets

ds_bronze_smartcity

ds_raw_smartcity

ds_raw_traffic

ds_raw_weather

ds_traffic_api

ds_weather_api

Data flows

Power Query

Activities

Get Metadata

Get_Files

ForEach

ForEach1

Activities

Copy data1

Parameters

Variables

Settings

Output

Pipeline run ID fe4aff0f-53ab-4ef5-be8a-6fbfb3af878

Pipeline status Succeeded

View debug run consumption

Monitor in Azure Metrics

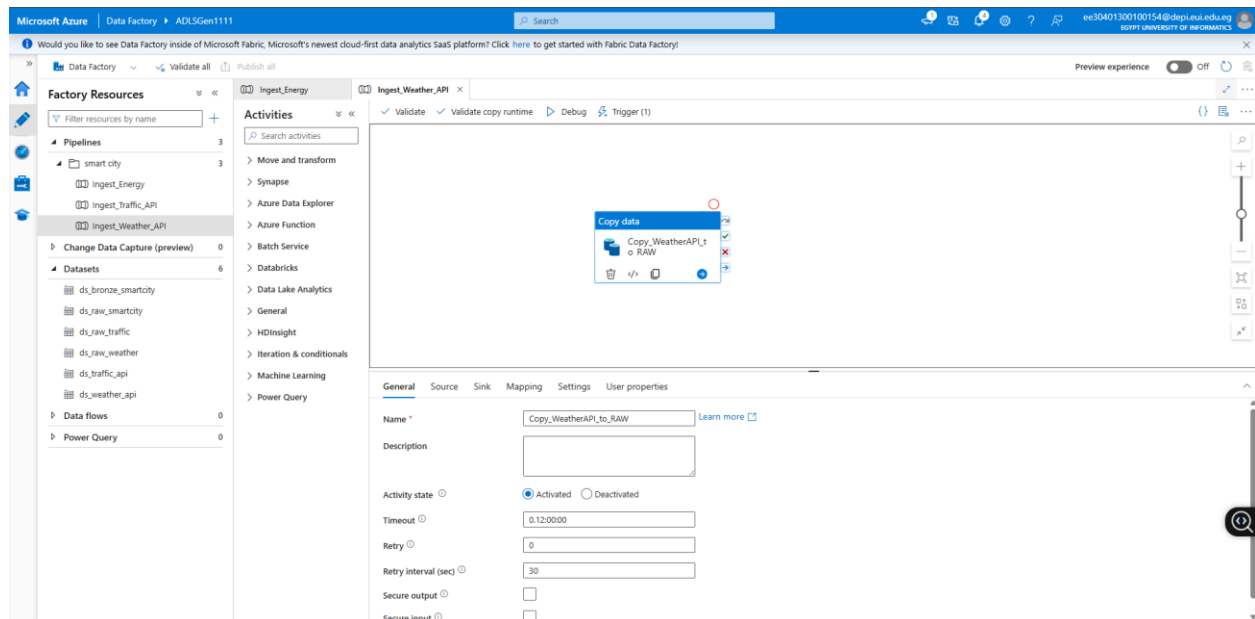
Export to CSV

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime	User prop...	Activity run ID
Copy data1	Succeeded	Copy data	11/29/2025, 11:18:24 PM	15s	AutoResolveIntegrationRuntime (France Central)		01a57912-0445-4fa7-ad12-bd9d
Copy data1	Succeeded	Copy data	11/29/2025, 11:18:11 PM	13s	AutoResolveIntegrationRuntime (France Central)		bad7c72-883d-41ea-9f57-b51e
Copy data1	Succeeded	Copy data	11/29/2025, 11:17:58 PM	12s	AutoResolveIntegrationRuntime (France Central)		957c1c9b-8e5a-4587-b984-3d97a
Copy data1	Succeeded	Copy data	11/29/2025, 11:17:45 PM	13s	AutoResolveIntegrationRuntime (France Central)		bcd23a3c-5d06-4784-89b7-b3ba2
Copy data1	Succeeded	Copy data	11/29/2025, 11:17:32 PM	12s	AutoResolveIntegrationRuntime (France Central)		93f62301-7d11-4250-a807-72ad9e
ForEach1	Succeeded	ForEach	11/29/2025, 11:17:31 PM	1m 9s			34d5fc69-092c-4b7b-84d0-00640

Pipeline 2 — Ingest Weather API (REST → Raw → Bronze)

Objective

Fetch real-time or daily weather data from a public API and store the JSON output into ADLS raw for further transformation in Silver.



Copy Data source/sink configuration

General	Source	Sink	Mapping	Settings	User properties
Source dataset * ds_weather_api Open New Preview data Learn more Request method * ⓘ GET Additional headers ⓘ Request body ⓘ Request timeout ⓘ Max concurrent connections ⓘ 					

General **Source** Sink Mapping Settings User properties

Request body ⓘ

Request timeout ⓘ

Max concurrent connections ⓘ

Additional columns ⓘ

+ New | Delete

Name	Value
load_date	@utcNow()
api_status	Custom success

General Source **Sink** Mapping Settings User properties

Sink dataset * [Open](#) [+ New](#) [Learn more](#)

Dataset properties ⓘ

Name	Value
raw_weather	@concat('weather_', formatDateTime...

Copy behavior ⓘ

Max concurrent connections ⓘ

Block size (MB) ⓘ

Metadata ⓘ [+ New](#)

REST dataset settings : URL (<https://api.open-meteo.com/v1/forecast>)

Ingest_Energy Ingest_Weather_API **ds_weather_api** × ds_raw_smartcity

JSON **ds_weather_api**

Connection Schema Parameters

Linked service * [Test connection](#) [Edit](#) [+ New](#)

Base URL

Relative URL ⓘ [Preview data](#)

Compression type

Encoding

Trigger details

first data analytics SaaS platform? Click [here](#) to get started with Fabric Data Factory!

Ingest_Weather_API x ds_weather_api ds_raw_smartcity

✓ Validate ✓ Validate copy runtime ▶ Debug ⚙ Trigger (1)

Copy data

Copy_WeatherAPI_t
o RAW

General Source Sink Mapping Settings User properties

Source dataset * ds_weather_api Open + New Preview

Request method * GET

Additional headers

Request body

Request timeout

Max concurrent connections

Edit trigger

Name * Daily_Weather_Trigger

Description

Type * ScheduleTrigger

Start date * 11/23/2025, 9:22:00 AM

Time zone * Cairo (UTC+2)

Recurrence * Every 1 Hour(s)

☐ Specify an end date

Annotations

+ New

Status ☐ Started ☒ Stopped

OK Cancel

Successful pipeline run

Microsoft Azure Data Factory ADLSGen111

Would you like to see Data Factory inside of Microsoft Fabric, Microsoft's newest cloud-first data analytics SaaS platform? Click [here](#) to get started with Fabric Data Factory!

Factory Resources

- Pipelines 3
 - smart city 3
 - Ingest_Energy
 - Ingest_Traffic_API
 - Ingest_Weather_API
- Datasets 6
 - ds_bronze_smartcity
 - ds_raw_smartcity
 - ds_raw_traffic
 - ds_raw_weather
 - ds_traffic_api
 - ds_weather_api
- Data flows 0
- Power Query 0

Activities

- Move and transform
- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service
- Databricks
- Data Lake Analytics
- General
- HDInsight
- Iteration & conditionals
- Machine Learning
- Power Query

Copy data

Copy_WeatherAPI_t
o RAW

Pipeline run ID: 749b96d6-aeb3-4536-a1f7-13e14bb3cee7 Pipeline status: Succeeded

View debug run consumption

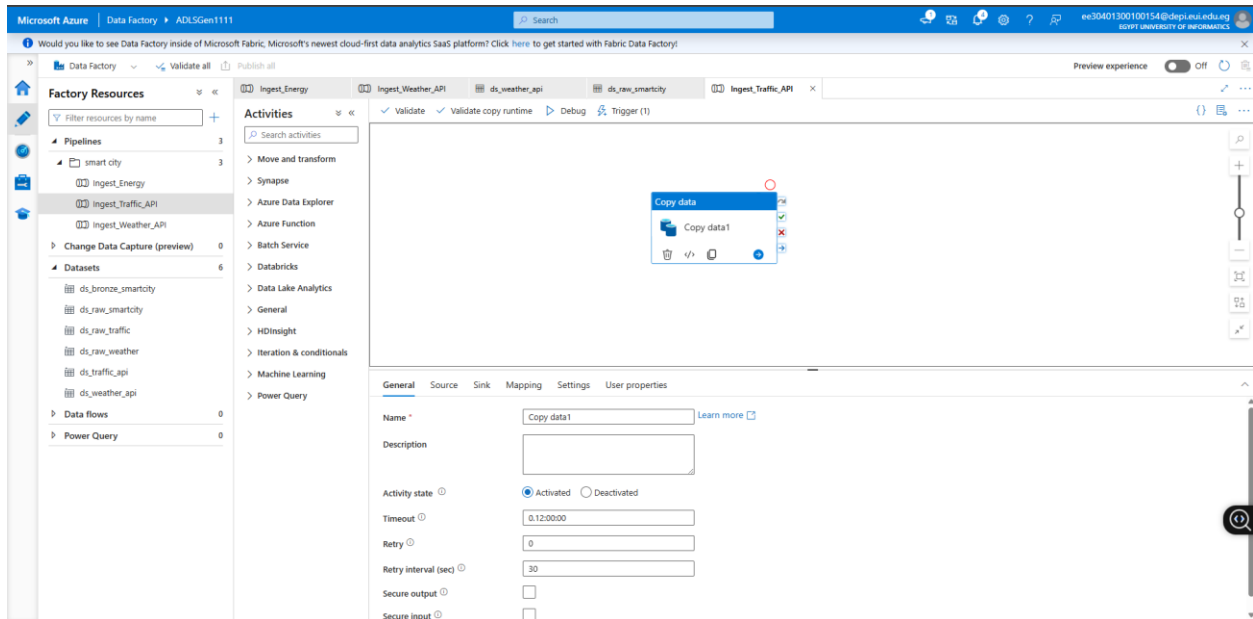
Monitor in Azure Metrics Export to CSV

Activity name	Activity status	Activity name	Run start	Duration	Integration runtime	User properties	Activity run ID
Copy_WeatherAPI_to_RAW	Succeeded	Copy data	11/29/2025, 11:46:52 PM	19s	AutoResolveIntegrationRuntime (France Central)		8079734-a4e3-4ab4-a836-a1f5

Pipeline 3 — Ingest Traffic API

Objective

Extract live traffic congestion information (speed, congestion index, city zone, etc.) and save it into ADLS raw for use in feature engineering and forecasting.



Copy Data source/sink configuration

General	Source	Sink	Mapping	Settings	User properties
Source dataset * ds_traffic_api Open New Preview data Learn more					
Request method * ⓘ GET					
Additional headers ⓘ					
Request body ⓘ					
Request timeout ⓘ					
Max concurrent connections ⓘ					

General **Source** Sink Mapping Settings User properties

Request body ⓘ

Request timeout ⓘ

Max concurrent connections ⓘ

Additional columns ⓘ

+ New | Delete

☐ Name	Value
☐ load_date	@utcNow()
☐ api_status	Custom ▼ success

General Source **Sink** Mapping Settings User properties

Sink dataset * ds_raw_traffic Open + New Learn more

Dataset properties ⓘ

Name	Value
raw_traffic	@concat('traffic_', formatDateTime(u...

Copy behavior ⓘ Select...

Max concurrent connections ⓘ

Block size (MB) ⓘ

Metadata ⓘ + New

REST dataset settings : URL(<https://api.tomtom.com>)

On Azure, Microsoft's cloud data analytics platform, click [here](#) to get started with Azure Data Factory.

1 Publish all

Ingest_Energy Ingest_Weather_API ds_weather_api ds_raw_smartcity Ingest_Traffic_API ds_traffic_api

JSON ds_traffic_api

Connection Schema Parameters

Linked service * LS_TrafficAPI Test connection Edit + New Learn more

Base URL https://api.tomtom.com

Relative URL ⓘ traffic/services/4/flowSegmentData/abso... Preview data

Compression type No compression

Encoding Default(UTF-8)

Trigger details

Edit trigger

Name *
traffic

Description

Type *
ScheduleTrigger

Start date * ⓘ
11/23/2025, 5:18:00 PM

Time zone * ⓘ
Cairo (UTC+2)

Recurrence * ⓘ
Every 1 Hour(s)

☐ Specify an end date

Annotations
+ New

Status ⓘ
☐ Started ☒ Stopped

OK Cancel

Successful pipeline run

Microsoft Azure | Data Factory | ADXSGen1111

Would you like to see Data Factory inside of Microsoft Fabric, Microsoft's newest cloud-first data analytics SaaS platform? Click here to get started with Fabric Data Factory!

Factory Resources

- Pipelines (3)
- smart city (3)
- ds_bronze_smartcity
- ds_raw_smartcity
- ds_raw_traffic
- ds_raw_weather
- ds_traffic_api
- ds_weather_api
- Data flows (0)
- Power Query (0)

Activities

- Move and transform
- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service
- Databricks
- Data Lake Analytics
- General
- HDInsight
- Iteration & conditionals
- Machine Learning
- Power Query

Copy data

Copy data1

Parameters Variables Settings Output

Pipeline run ID: 06b811a1-1876-49b1-9dbb-21fc286548f3 Pipeline status: Succeeded

View debug run consumption

Monitor in Azure Metrics | Export to CSV

Activity name	Activity status	Activity name	Run start	Duration	Integration runtime	User prop...	Activity run ID
Copy data1	Succeeded	Copy data	11/20/2025, 12:01:31 AM	14s	AutoResolveIntegrationRuntime (France Central)		9844d986-1023-4d68-94b4-145d

ADF Monitoring & Alerts

Monitoring

ADF Monitor is used to track:

- Pipeline runs
- Activity runs
- Trigger executions
- Failures & errors
- Retry attempts
- API response failures

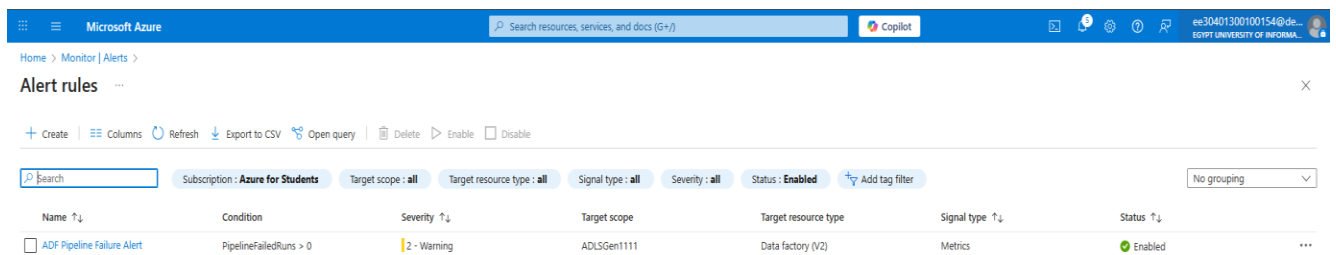
Pipeline name	Run start	Run end	Duration	Triggered by	Status	Run	Parameters	Annotations	Run ID
pipeline1	11/23/2025, 5:56:06 AM	11/23/2025, 5:56:08 AM	3s	Manual trigger	Failed	Original			7f5144e7-7fed-47ca-9481-d1af5d59e
pipeline1	11/23/2025, 12:47:00 PM	11/23/2025, 12:48:14 PM	1m 14s	Daily_Files_Trigger	Succeeded	Original			58f0666-955d-45a7-ba5e-612a5333c
pipeline1	11/23/2025, 1:47:01 PM	11/23/2025, 1:48:11 PM	1m 10s	Daily_Files_Trigger	Succeeded	Original			38747feb-cc98-4838-bb20-bcd706ae
pipeline1	11/23/2025, 2:47:01 PM	11/23/2025, 2:48:16 PM	1m 16s	Daily_Files_Trigger	Succeeded	Original			91fde051-dbe7-4437-b0ed-48665fb2
pipeline1	11/23/2025, 3:47:01 PM	11/23/2025, 3:48:13 PM	1m 13s	Daily_Files_Trigger	Succeeded	Original			1656f469-82da-4b60-9ab5-d95856ff
pipeline1	11/23/2025, 4:47:02 PM	11/23/2025, 4:48:15 PM	1m 14s	Daily_Files_Trigger	Succeeded	Original			9e8294cf-9eae-4e1e-8d01-7df4c315
pipeline1	11/23/2025, 5:47:00 PM	11/23/2025, 5:48:13 PM	1m 14s	Daily_Files_Trigger	Succeeded	Original			2904e11f-72a3-4110-a307-d1bb4f86
pipeline1	11/23/2025, 6:47:00 PM	11/23/2025, 6:48:19 PM	1m 19s	Daily_Files_Trigger	Succeeded	Original			00e06636-0482-4823-8d08-3a4fe153
pipeline1	11/23/2025, 7:47:00 PM	11/23/2025, 7:48:25 PM	1m 25s	Daily_Files_Trigger	Succeeded	Original			1fa4ea95-c417-438f-b957-8a826cab6
Ingest_Traffic_API	11/23/2025, 8:18:01 PM	11/23/2025, 8:18:18 PM	18s	traffic	Succeeded	Original			3157f83c-75cc-4ece-ba1d-5ad5d4c1
pipeline1	11/23/2025, 8:47:00 PM	11/23/2025, 8:48:44 PM	1m 44s	Daily_Files_Trigger	Failed	Original			749b349a-3894-4184-a548-cfd305ef
Ingest_Traffic_API	11/23/2025, 9:18:00 PM	11/23/2025, 9:18:16 PM	17s	traffic	Succeeded	Original			320c6dd5-c7af-4102-9a43-9a585a41
pipeline1	11/23/2025, 9:47:00 PM	11/23/2025, 9:49:01 PM	2m 2s	Daily_Files_Trigger	Failed	Original			e65a4678-04cb-471b-8430-b1de8f4a
Ingest_Traffic_API	11/23/2025, 10:18:00 PM	11/23/2025, 10:18:18 PM	19s	traffic	Succeeded	Original			8d0feaf6-caee-49f4-9b85-31b6d22ac
pipeline1	11/23/2025, 10:47:00 PM	11/23/2025, 10:49:32 PM	2m 32s	Daily_Files_Trigger	Failed	Original			072ae1c9-7c3d-49a6-bf68-58cc9859
Ingest_Traffic_API	11/23/2025, 11:18:01 PM	11/23/2025, 11:18:18 PM	17s	traffic	Succeeded	Original			d08d7ec3-35f0-4c46-b0cb-9f8cae0d
pipeline1	11/23/2025, 11:47:01 PM	11/23/2025, 11:49:40 PM	2m 40s	Daily_Files_Trigger	Failed	Original			71fcb81c-4e68-4816-a23c-949e022
Ingest_Traffic_API	11/24/2025, 12:18:00 AM	11/24/2025, 12:18:20 AM	20s	traffic	Succeeded	Original			5eead018-b455-4940-9e78-36726151
pipeline1	11/24/2025, 12:47:00 AM	11/24/2025, 12:49:48 AM	2m 49s	Daily_Files_Trigger	Failed	Original			67a9ee94-83da-4ae2-96f9-f58a99ee
Ingest_Weather_API	11/24/2025, 1:00:06 AM	11/24/2025, 1:00:28 AM	22s	Daily_Weather_Trigger	Succeeded	Original			ec648816-04b5-48da-a800-4a2c2481
Ingest_Traffic_API	11/24/2025, 1:18:01 AM	11/24/2025, 1:18:17 AM	17s	traffic	Succeeded	Original			260e71e1-9c8f-4771-965c-9d98c2e41

Alerts (Azure Monitor Integration)

Pipeline Failure

Alert includes:

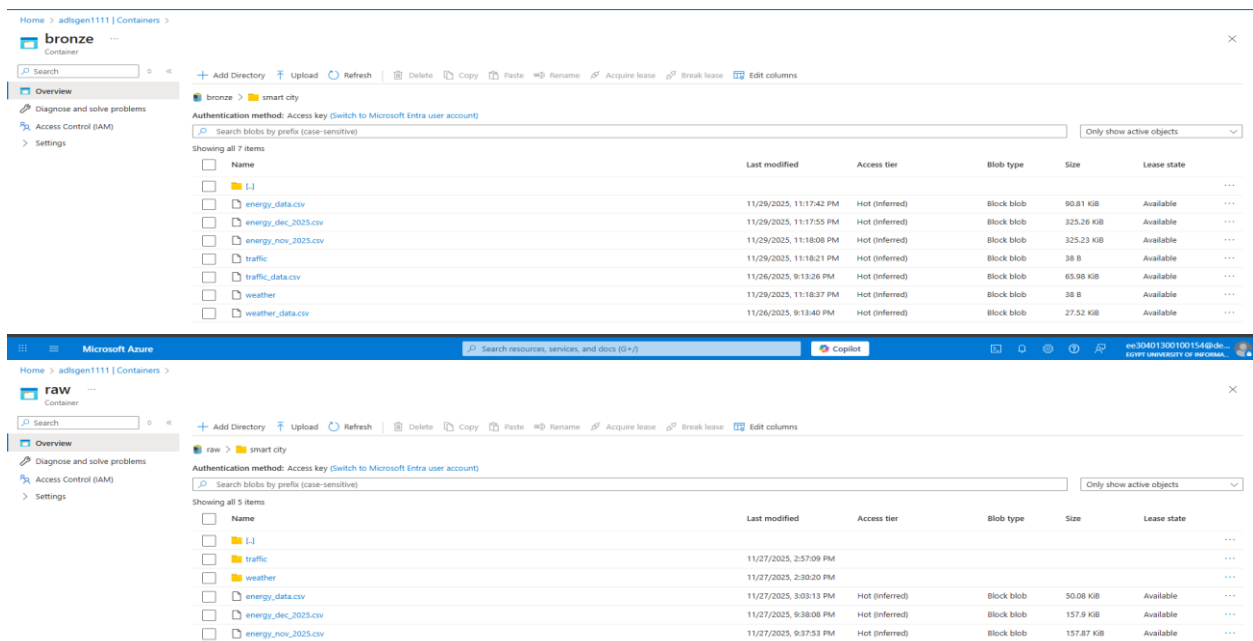
- Pipeline name
- Error message
- Severity level
- Timestamp
- Email notification to the team



The screenshot shows the 'Alert rules' page in the Microsoft Azure portal. The alert rule is named 'ADF Pipeline Failure Alert' and is currently enabled. The condition is 'PipelineFailedRuns > 0'. The target scope is 'ADLSGen1111' and the target resource type is 'Data factory (V2)'. The signal type is 'Metrics' and the status is 'Enabled'.

Name	Condition	Severity	Target scope	Target resource type	Signal type	Status
ADF Pipeline Failure Alert	PipelineFailedRuns > 0	2 - Warning	ADLSGen1111	Data factory (V2)	Metrics	Enabled

Out puts in raw and bronze continuers

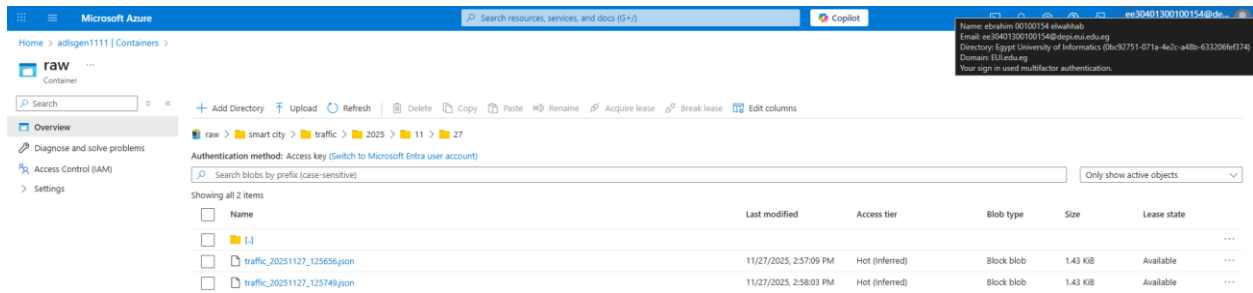


The screenshots show the 'Overview' page of the 'bronze' and 'raw' containers in the Microsoft Azure portal. The 'bronze' container shows 7 items, including 'energy_data.csv', 'energy_dec_2025.csv', 'energy_nov_2025.csv', 'traffic', 'traffic_data.csv', 'weather', and 'weather_data.csv'. The 'raw' container shows 5 items, including 'traffic', 'weather', 'energy_data.csv', 'energy_dec_2025.csv', and 'energy_nov_2025.csv'.

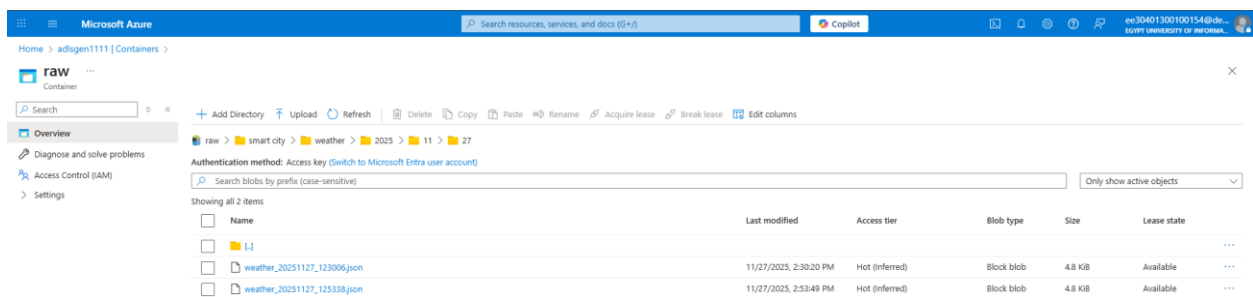
Name	Last modified	Access tier	Blob type	Size	Lease state
[-]					...
energy_data.csv	11/29/2025, 11:17:42 PM	Hot (inferred)	Block blob	90.81 KiB	Available
energy_dec_2025.csv	11/29/2025, 11:17:55 PM	Hot (inferred)	Block blob	325.26 KiB	Available
energy_nov_2025.csv	11/29/2025, 11:18:08 PM	Hot (inferred)	Block blob	325.23 KiB	Available
traffic	11/29/2025, 11:18:21 PM	Hot (inferred)	Block blob	38 B	Available
traffic_data.csv	11/26/2025, 9:13:26 PM	Hot (inferred)	Block blob	65.98 KiB	Available
weather	11/29/2025, 11:18:37 PM	Hot (inferred)	Block blob	38 B	Available
weather_data.csv	11/26/2025, 9:13:40 PM	Hot (inferred)	Block blob	27.52 KiB	Available

Name	Last modified	Access tier	Blob type	Size	Lease state
[-]					...
traffic	11/27/2025, 2:57:09 PM				...
weather	11/27/2025, 2:30:20 PM				...
energy_data.csv	11/27/2025, 3:03:13 PM	Hot (inferred)	Block blob	50.08 KiB	Available
energy_dec_2025.csv	11/27/2025, 9:38:08 PM	Hot (inferred)	Block blob	157.9 KiB	Available
energy_nov_2025.csv	11/27/2025, 9:37:53 PM	Hot (inferred)	Block blob	157.87 KiB	Available

Traffic Json files



Weather json files



4. Silver Layer — Databricks Processing

The Silver Layer represents the cleaned, standardized, and enriched version of the Raw/Bronze data.

In this layer, we apply data quality rules, feature engineering, and finally create a unified master-table combining Energy, Weather, and Traffic information.

1) Create notebooks

in this layer we used 4 notebooks on databricks

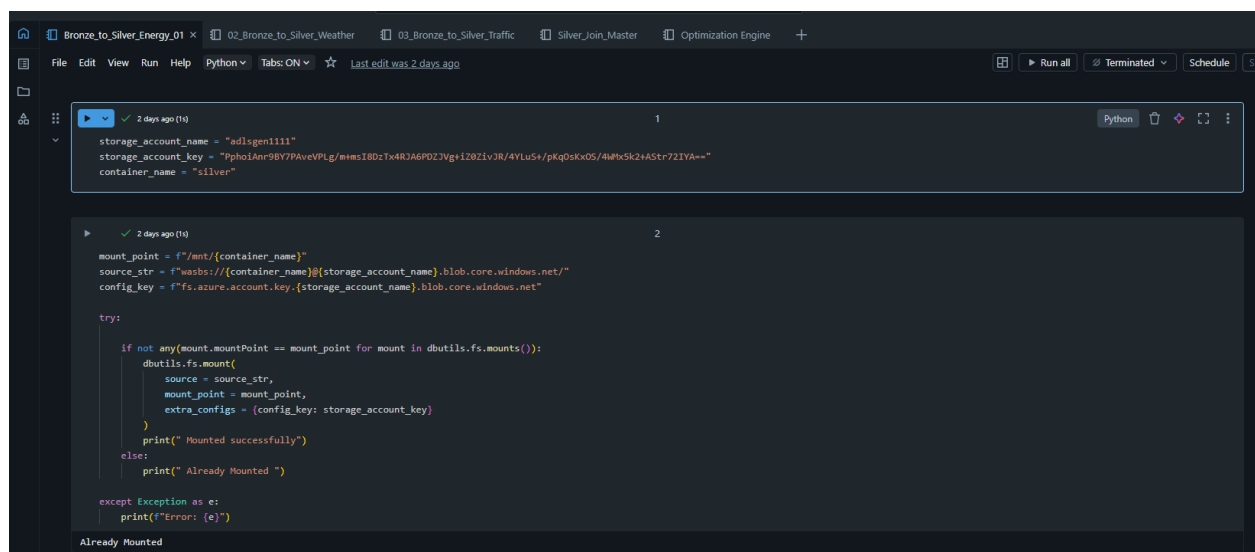
1. Bronze_to_Silver_Energy
2. Bronze_to_Silver_Weather
3. Bronze_to_Silver_Traffic
4. Silver_Join_Master

• in first 3 notebooks we did :

1. Mount ADLS to workspace
2. Load Bronze tables in csv or delta
3. Schema validation
4. Data Cleaning Logic:
 - 4.1 Remove Duplicates
 - 4.2 Fix timestamps
 - 4.3 String Standardization
 - 4.4 Missing Value Handling
5. Feature Engineering:
 - 5.1 peak_hour_flag
 - 5.2 weekend_flag
 - 5.3 weather_index
 - 5.4 congestion_index

1. Bronze_to_Silver_Energy

1. Mount ADLS to workspace



The screenshot shows a Jupyter Notebook with two code cells. The first cell defines variables for storage account name, key, and container name. The second cell contains logic to mount the ADLS container to the workspace, checking if it's already mounted and printing success or error messages.

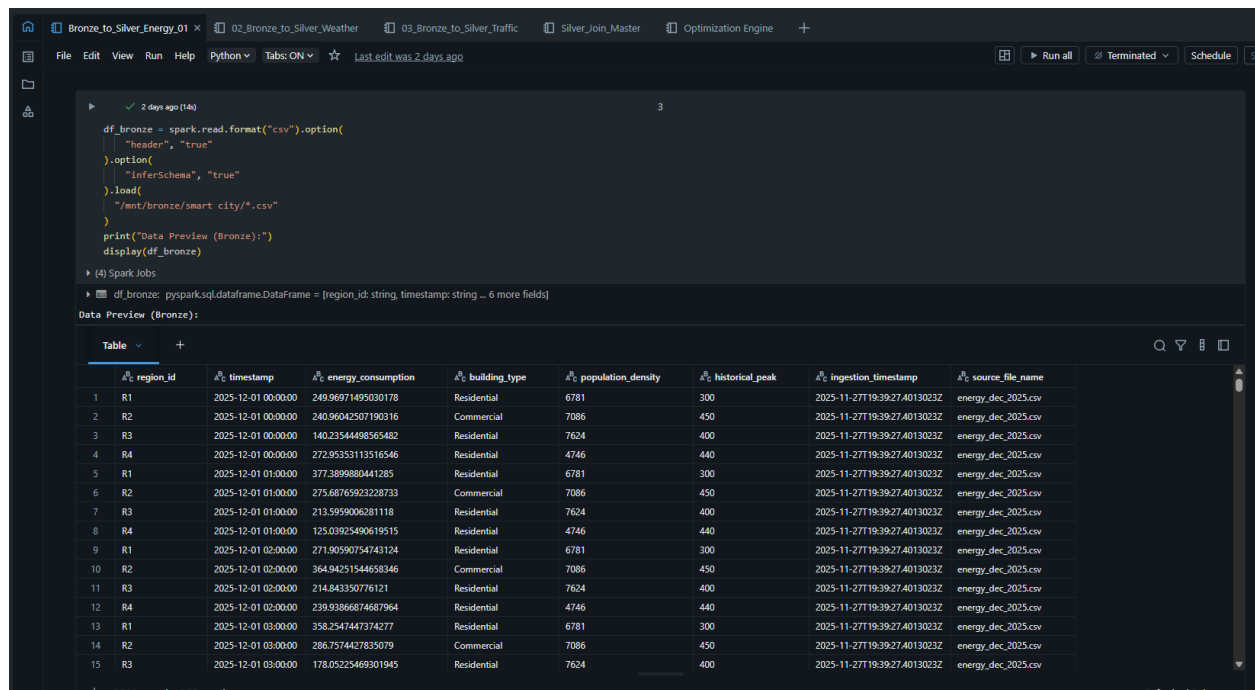
```
storage_account_name = "adlsgen1111"
storage_account_key = "Ppho1Anr9BY7PAveVPLg/mHms18DzTx4R3AGPDZ3Vgti202iv3R/4YLu5+/pKq0sKx05/4W5k2iA5tr72IYA=="
container_name = "silver"

mount_point = f"/mnt/{container_name}"
source_str = f"wasbs://{container_name}@{storage_account_name}.blob.core.windows.net/"
config_key = f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net"

try:
    if not any(mount.mountPoint == mount_point for mount in dutils.fs.mounts()):
        dutils.fs.mount(
            source = source_str,
            mount_point = mount_point,
            extra_configs = (config_key, storage_account_key)
        )
        print("Mounted successfully")
    else:
        print("Already Mounted ")
except Exception as e:
    print(f"Error: {e}")
```

Already Mounted

2. Load Bronze tables in csv or delta and Schema validation



```
df_bronze = spark.read.format("csv").option(
    "header", "true"
).option(
    "inferSchema", "true"
).load(
    "/mnt/bronze/smart city/*.csv"
)
print("Data Preview (Bronze):")
display(df_bronze)
```

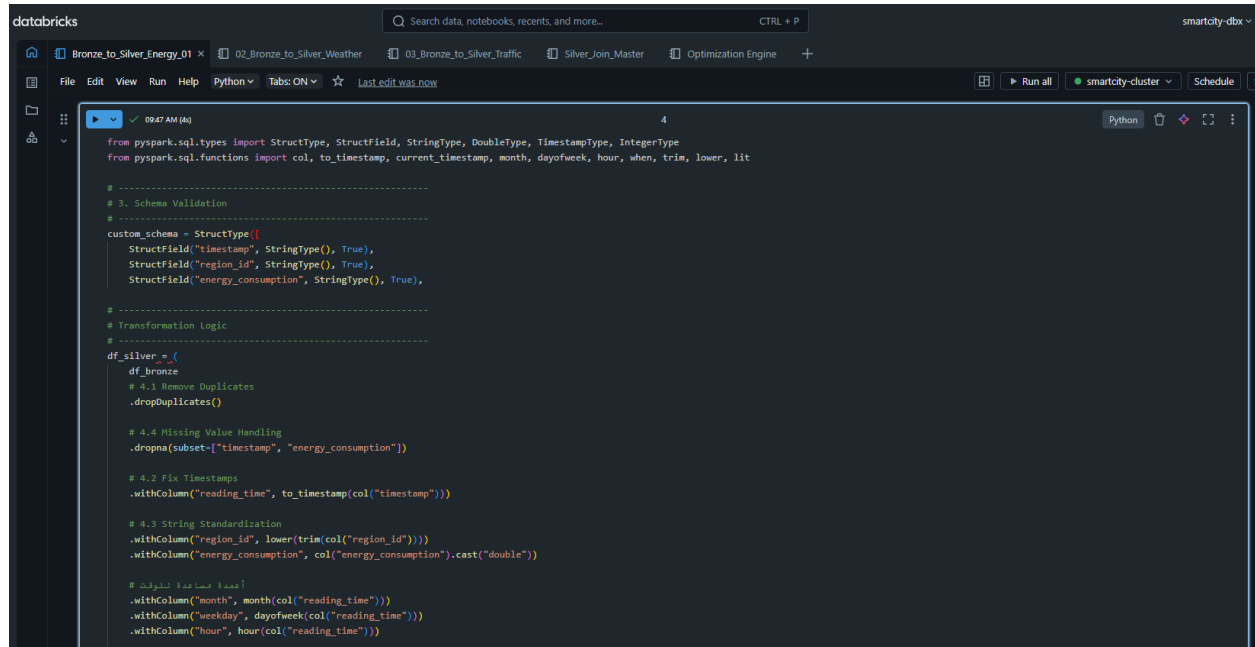
(4) Spark Jobs

df_bronze: pyspark.sql.dataframe.DataFrame = [region_id: string, timestamp: string ... 6 more fields]

Data Preview (Bronze):

	region_id	timestamp	energy_consumption	building_type	population_density	historical_peak	ingestion_timestamp	source_file_name
1	R1	2025-12-01 00:00:00	249.96971495030178	Residential	6781	300	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
2	R2	2025-12-01 00:00:00	240.96042507190316	Commercial	7086	450	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
3	R3	2025-12-01 00:00:00	140.23544498565482	Residential	7624	400	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
4	R4	2025-12-01 00:00:00	272.95353113516546	Residential	4746	440	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
5	R1	2025-12-01 01:00:00	377.3899880441285	Residential	6781	300	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
6	R2	2025-12-01 01:00:00	275.68765923228733	Commercial	7086	450	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
7	R3	2025-12-01 01:00:00	213.5959006281118	Residential	7624	400	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
8	R4	2025-12-01 01:00:00	125.03825490619515	Residential	4746	440	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
9	R1	2025-12-01 02:00:00	271.90590734743124	Residential	6781	300	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
10	R2	2025-12-01 02:00:00	364.94251544658346	Commercial	7086	450	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
11	R3	2025-12-01 02:00:00	214.843350776121	Residential	7624	400	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
12	R4	2025-12-01 02:00:00	239.93866874687964	Residential	4746	440	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
13	R1	2025-12-01 03:00:00	358.2547447374277	Residential	6781	300	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
14	R2	2025-12-01 03:00:00	286.7574427835079	Commercial	7086	450	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv
15	R3	2025-12-01 03:00:00	178.05225469301945	Residential	7624	400	2025-11-27T19:39:27.4013023Z	energy_dec_2025.csv

3. Data Cleaning Logic & Feature Engineering:



```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, TimestampType, IntegerType
from pyspark.sql.functions import col, to_timestamp, current_timestamp, month, dayofweek, hour, when, trim, lower, lit

# 3. Schema Validation
# -----
custom_schema = StructType([
    StructField("timestamp", StringType(), True),
    StructField("region_id", StringType(), True),
    StructField("energy_consumption", StringType(), True),
])

# Transformation Logic
# -----
df_silver = (
    df_bronze
    # 4.1 Remove Duplicates
    .dropDuplicates()

    # 4.4 Missing Value Handling
    .dropna(subset=["timestamp", "energy_consumption"])

    # 4.2 Fix Timestamps
    .withColumn("reading_time", to_timestamp(col("timestamp")))

    # 4.3 String Standardization
    .withColumn("region_id", lower(trim(col("region_id"))))
    .withColumn("energy_consumption", col("energy_consumption").cast("double"))

    # 4.5 Date Feature Extraction
    .withColumn("month", month(col("reading_time")))
    .withColumn("weekday", dayofweek(col("reading_time")))
    .withColumn("hour", hour(col("reading_time")))
```

```
# 5. Feature Engineering
# -----

# 5.1 Peak Hours Flag
.withColumn("peak_hours_flag",
            when(col("hour") >= 17) & (col("hour") <= 22), 1).otherwise(0))

# 5.2 Weekend Flag
.withColumn("weekend_flag",
            when(col("weekday").isin([6, 7]), 1).otherwise(0))

.withColumn("processing_time", current_timestamp())
.select(
    "reading_time",
    "region_id",
    "energy_consumption",
    "month",
    "weekday",
    "hour",
    "peak_hours_flag",
    "weekend_flag", # New
    "processing_time"
)

print("Successfully transformed with all features.")
display(df_silver)
```

reading_time	region_id	energy_consumption	month	weekday	hour	peak_hours_flag	weekend_flag	processing_time
2025-12-01T19:00:00.000+00...	4	191.87507866761503	12	5	19	1	0	2025-11-30T07:47:27.667+00...

4. Save the the final table of energy in silver

```
ssave_path = "/mnt/silver/smart city/energy"
df_silver.write.format("delta").mode("overwrite").option("mergeSchema", "true").save(ssave_path)
print(f"saved to {save_path}")
```

saved to /mnt/silver/smart city/energy

2. Bronze_to_Silver_Weather

1. Mount ADLS to workspace

```
storage_account_name = "ndlsigen1111"
storage_account_key = "0pkoIAm96779AveVPLg/ewmsI8DzTw4R3A6PDZ3Vg+12021v3R/4YLu5+/pKq0sKx05/4WtX5k2+AStr72IYA=="
container_name = "silver"

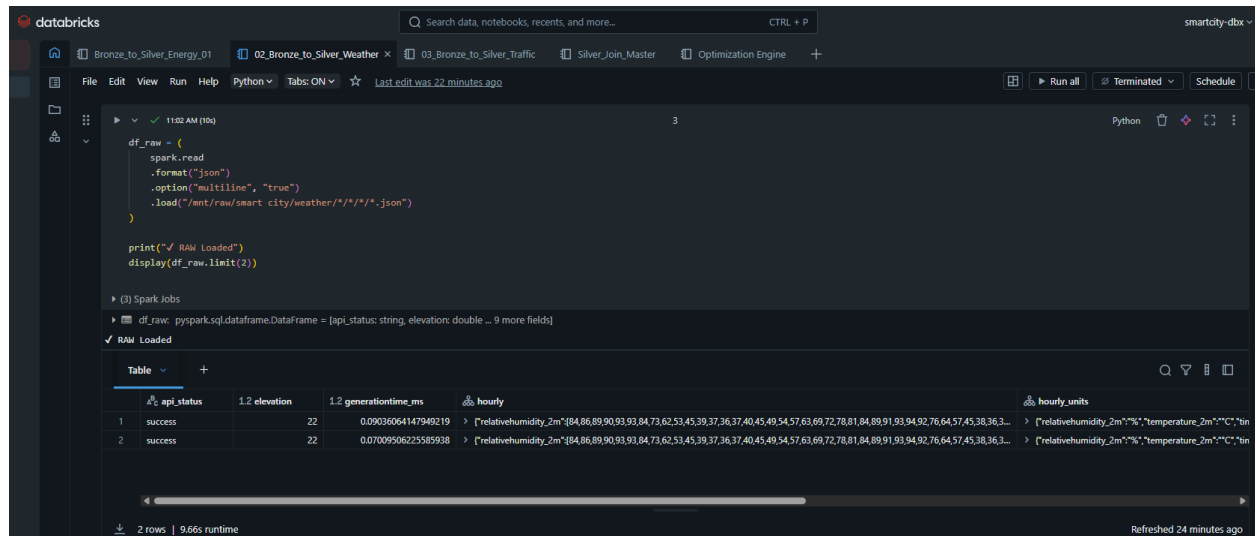
mount_point = f"/mnt/{container_name}"
source_str = f"wasbs://{container_name}@{storage_account_name}.blob.core.windows.net/"
config_key = f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net"

try:
    dbutils.fs.unmount(mount_point)
except:
    pass

try:
    dbutils.fs.mount(
        source = source_str,
        mount_point = mount_point,
        extra_configs = (config_key, storage_account_key)
    )
    print(f"Mounted successfully to {mount_point}")
except Exception as e:
    print(f"Error: {e}")

/mnt/silver has been unmounted.
Mounted successfully to /mnt/silver
```

2. Load Bronze tables in delta & Schema Validation



The screenshot shows a Databricks notebook with the following code and output:

```
df_raw = (
    spark.read
      .format("json")
      .option("multiline", "true")
      .load("/mnt/raw/smart_city/weather/**/*.json")
)

print("✓ RAW Loaded")
display(df_raw.limit(2))
```

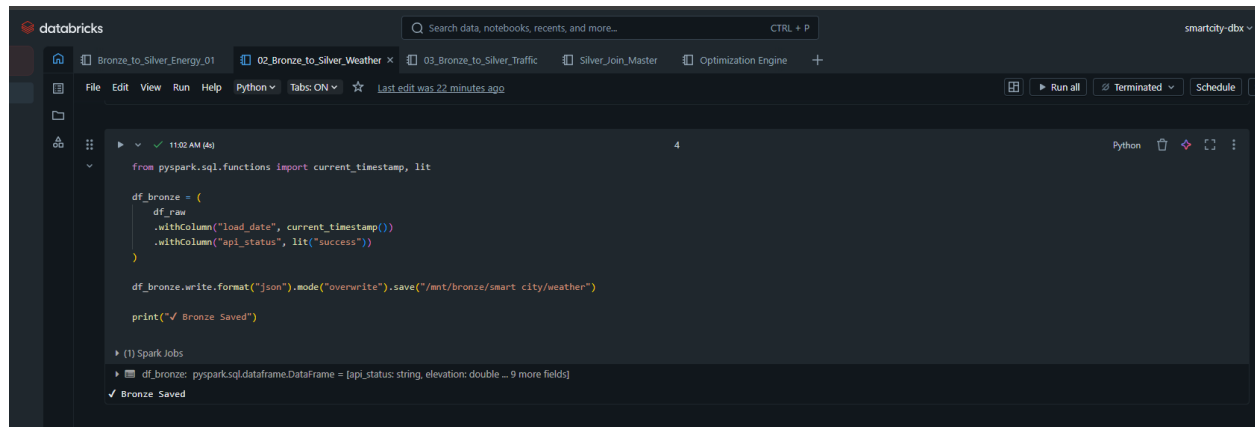
Output (3) Spark Jobs:

- df_raw: pyspark.sql.dataframe.DataFrame = [api_status: string, elevation: double ... 9 more fields]

✓ RAW Loaded

api_status	elevation	generationtime_ms	hourly	hourly_units
success	22	0.0903606147949219	[{"relativehumidity_2m":84.86,89.90,93.93,84.73,62.53,45.39,37.36,37.40,45.49,54.57,63.69,72.78,81.84,89.91,93.94,92.76,64.57,45.38,36.3...	[{"relativehumidity_2m":"%", "temperature_2m":"°C", "in
success	22	0.07009506225585938	[{"relativehumidity_2m":84.86,89.90,93.93,84.73,62.53,45.39,37.36,37.40,45.49,54.57,63.69,72.78,81.84,89.91,93.94,92.76,64.57,45.38,36.3...	[{"relativehumidity_2m":"%", "temperature_2m":"°C", "in

2 rows | 9.66s runtime | Refreshed 24 minutes ago



The screenshot shows a Databricks notebook with the following code and output:

```
from pyspark.sql.functions import current_timestamp, lit

df_bronze = (
    df_raw
      .withColumn("load_date", current_timestamp())
      .withColumn("api_status", lit("success"))
)

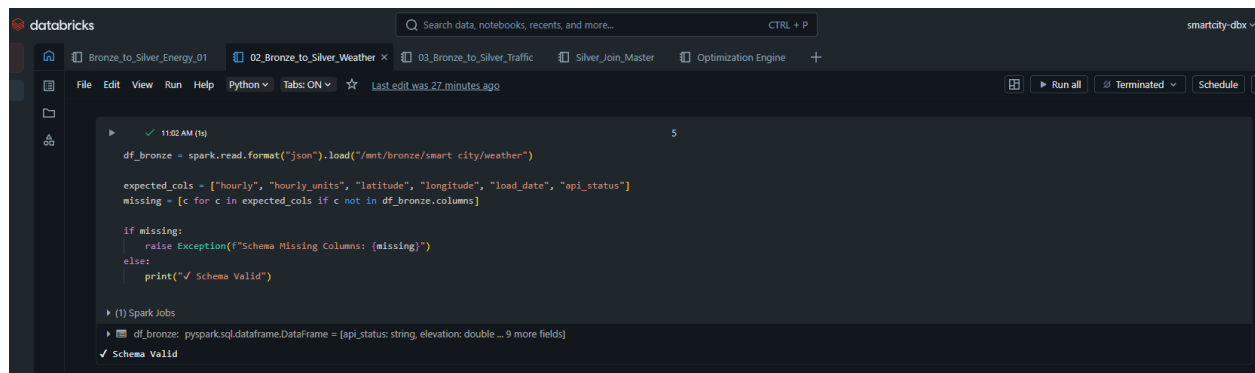
df_bronze.write.format("json").mode("overwrite").save("/mnt/bronze/smart_city/weather")

print("✓ Bronze Saved")
```

Output (1) Spark Jobs:

- df_bronze: pyspark.sql.dataframe.DataFrame = [api_status: string, elevation: double ... 9 more fields]

✓ Bronze Saved



The screenshot shows a Databricks notebook with the following code and output:

```
df_bronze = spark.read.format("json").load("/mnt/bronze/smart_city/weather")

expected_cols = ["hourly", "hourly_units", "latitude", "longitude", "load_date", "api_status"]
missing = [c for c in expected_cols if c not in df_bronze.columns]

if missing:
    raise Exception(f"Schema Missing Columns: {missing}")
else:
    print("✓ Schema Valid")
```

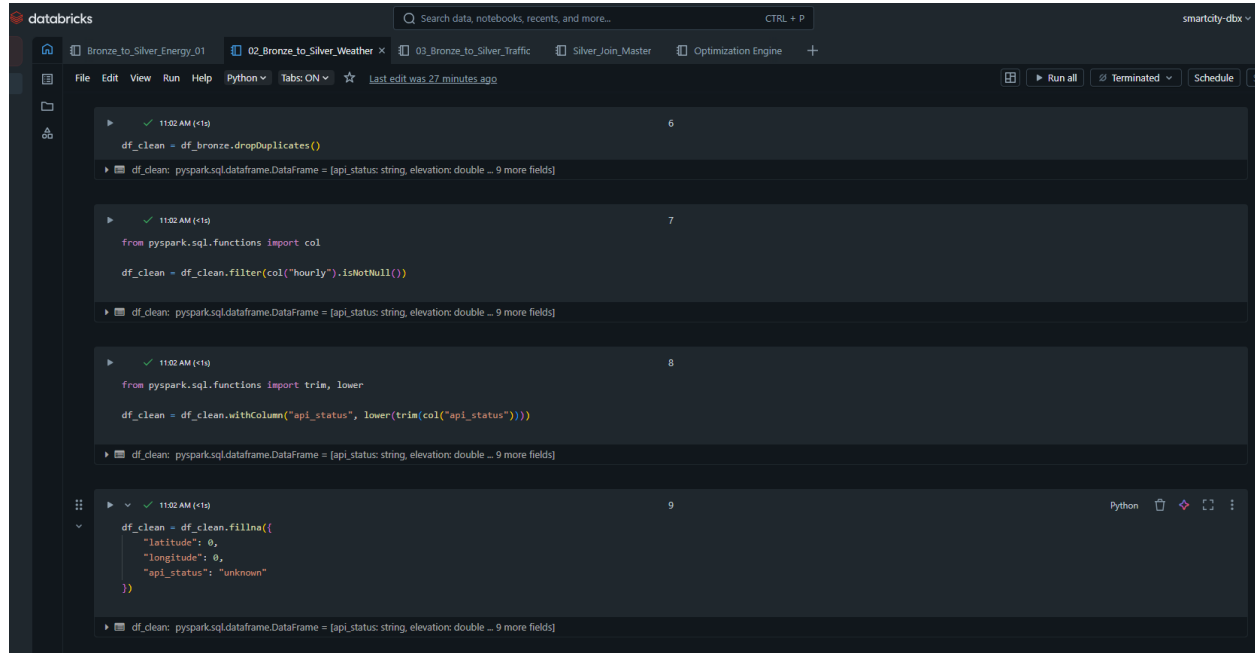
Output (1) Spark Jobs:

- df_bronze: pyspark.sql.dataframe.DataFrame = [api_status: string, elevation: double ... 9 more fields]

✓ Schema Valid

3. Data Cleaning Logic:

Remove Duplicates & Fix timestamp format + check hourly struct
Standardize strings & Missing Values



The screenshot shows a Databricks notebook with the following code cells:

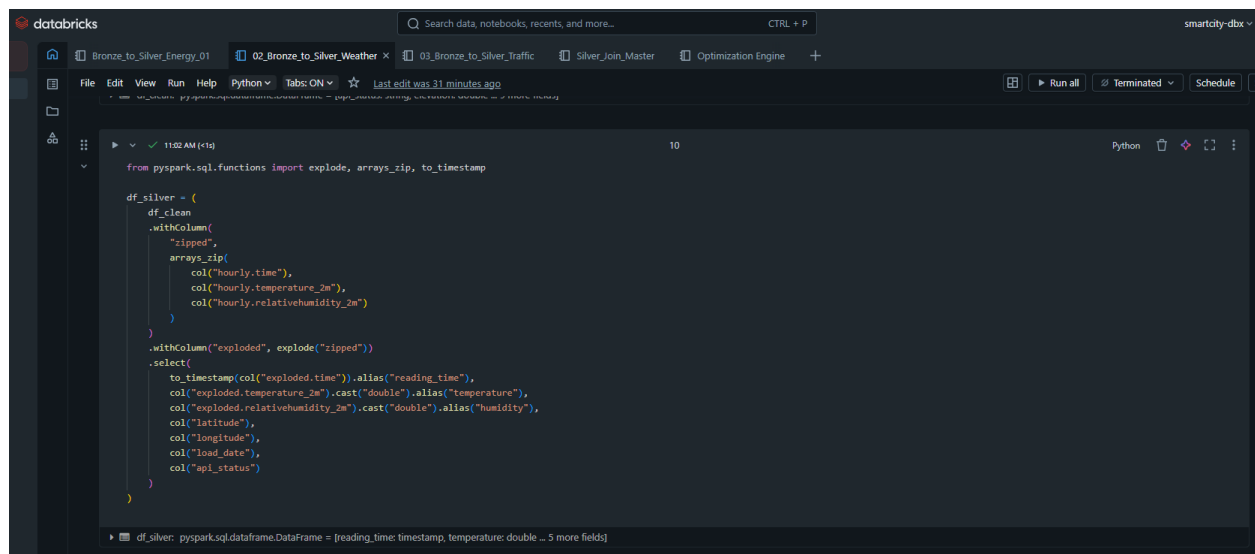
```
df_clean = df_bronze.dropDuplicates()
```

```
from pyspark.sql.functions import col
df_clean = df_clean.filter(col("hourly").isNotNull())
```

```
from pyspark.sql.functions import trim, lower
df_clean = df_clean.withColumn("api_status", lower(trim(col("api_status"))))
```

```
df_clean = df_clean.fillna({
  "latitude": 0,
  "longitude": 0,
  "api_status": "unknown"
})
```

4. silver Transformation

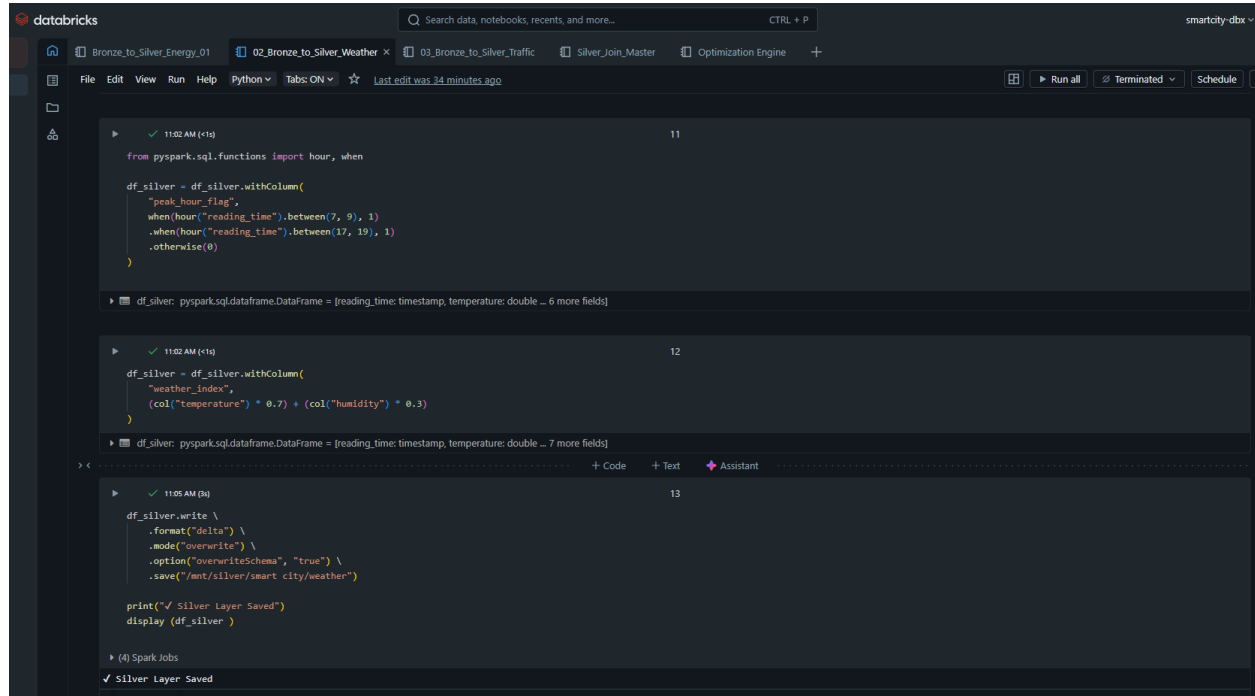


The screenshot shows a Databricks notebook with the following code cell:

```
from pyspark.sql.functions import explode, arrays_zip, to_timestamp

df_silver = (
  df_clean
    .withColumn(
      "zipped",
      arrays_zip(
        col("hourly.time"),
        col("hourly.temperature_2m"),
        col("hourly.relativehumidity_2m")
      )
    )
    .withColumn("exploded", explode("zipped"))
    .select(
      to_timestamp(col("exploded.time")).alias("reading_time"),
      col("exploded.temperature_2m").cast("double").alias("temperature"),
      col("exploded.relativehumidity_2m").cast("double").alias("humidity"),
      col("latitude"),
      col("longitude"),
      col("load_date"),
      col("api_status")
    )
)
```

5. Feature Engineering: Peak Hour Flag & Weather Index then SAVE SILVER



The screenshot shows a Databricks notebook with three tabs: '01_Bronze_to_Silver_Energy', '02_Bronze_to_Silver_Weather' (active), and '03_Bronze_to_Silver_Traffic'. The active tab contains Python code for feature engineering. The first cell (11) adds a 'peak_hour_flag' column to 'df_silver' based on the 'reading_time' hour. The second cell (12) adds a 'weather_index' column calculated as $(\text{temperature} \times 0.7) + (\text{humidity} \times 0.3)$. The third cell (13) saves the resulting 'df_silver' DataFrame to a Delta table in the '/mnt/silver/smart city/weather' path. The notebook interface includes a search bar, a toolbar with 'Run all', 'Terminated', and 'Schedule' buttons, and a sidebar with file explorer and recent files.

```
from pyspark.sql.functions import hour, when

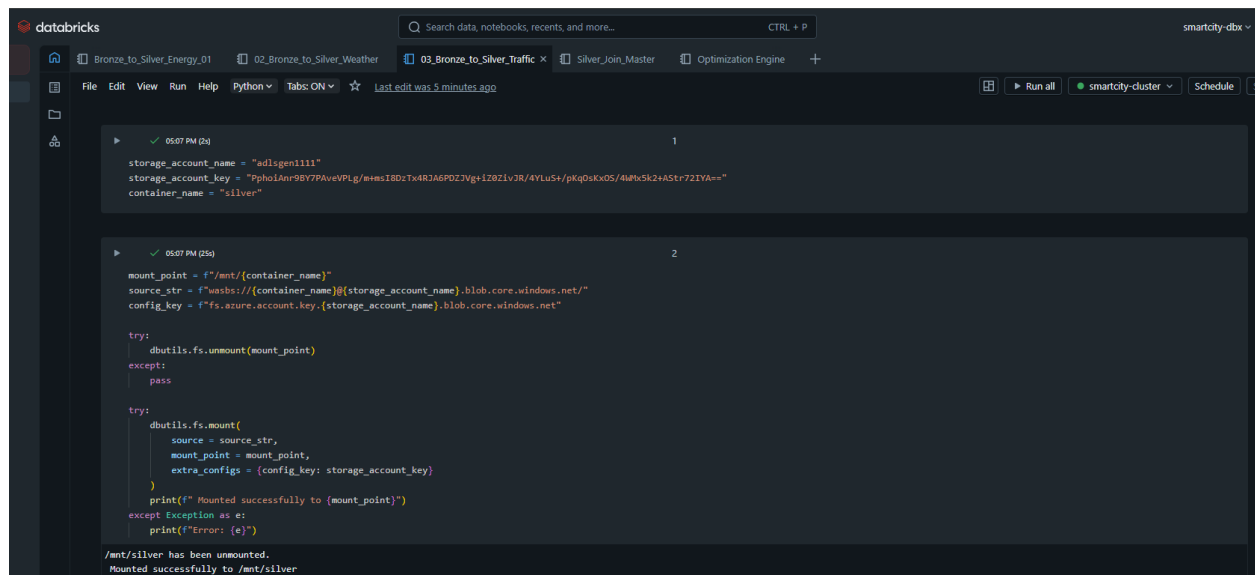
df_silver = df_silver.withColumn(
    "peak_hour_flag",
    when(hour("reading_time").between(7, 9), 1)
    .when(hour("reading_time").between(17, 19), 1)
    .otherwise(0)
)

df_silver = df_silver.withColumn(
    "weather_index",
    (col("temperature") * 0.7) + (col("humidity") * 0.3)
)

df_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save("/mnt/silver/smart city/weather")

print("✓ Silver Layer Saved")
display(df_silver)
```

3. Bronze_to_Silver_Traffic 1. Mount ADLS to workspace



The screenshot shows a Databricks notebook with four tabs: 'Bronze_to_Silver_Energy_01', '02_Bronze_to_Silver_Weather', '03_Bronze_to_Silver_Traffic' (active), and 'Silver_Join_Master'. The active tab contains Python code for mounting an Azure Data Lake Storage (ADLS) container. The first cell (1) defines the storage account name, key, and container name. The second cell (2) mounts the container to the workspace path '/mnt/silver'. The notebook interface includes a search bar, a toolbar with 'Run all', 'smartcity-cluster', and 'Schedule' buttons, and a sidebar with file explorer and recent files.

```
storage_account_name = "adlsgen1111"
storage_account_key = "PphoiAnr98V7PAveVPLg/wmsI8DzTx4R3A6PDZ3Vg+I28Zlv3R/4YLuS+pkqQsKx0S/4lWk5k24ASr72ZIVAm="
container_name = "silver"

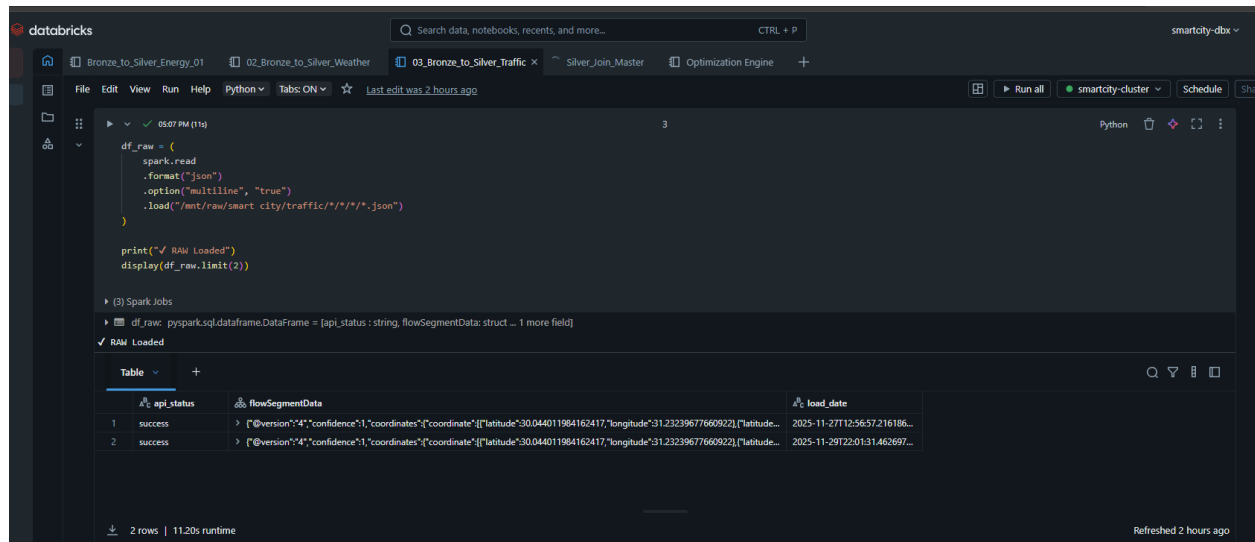
mount_point = f"/mnt/{container_name}"
source_str = f"wasbs://{container_name}@{storage_account_name}.blob.core.windows.net/"
config_key = f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net"

try:
    dbutils.fs.unmount(mount_point)
except:
    pass

try:
    dbutils.fs.mount(
        source = source_str,
        mount_point = mount_point,
        extra_configs = {config_key: storage_account_key}
    )
    print(f"Mounted successfully to {mount_point}")
except Exception as e:
    print(f"Error: {e}")

/mnt/silver has been unmounted.
Mounted successfully to /mnt/silver
```

2. Load Bronze tables in delta & Schema Validation



The screenshot shows a Databricks notebook with the following code and output:

```
df_raw = (
    spark.read
      .format("json")
      .option("multiline", "true")
      .load("/mnt/raw/smart-city/traffic/*/*/*.json")
)

print("✓ RAW Loaded")
display(df_raw.limit(2))
```

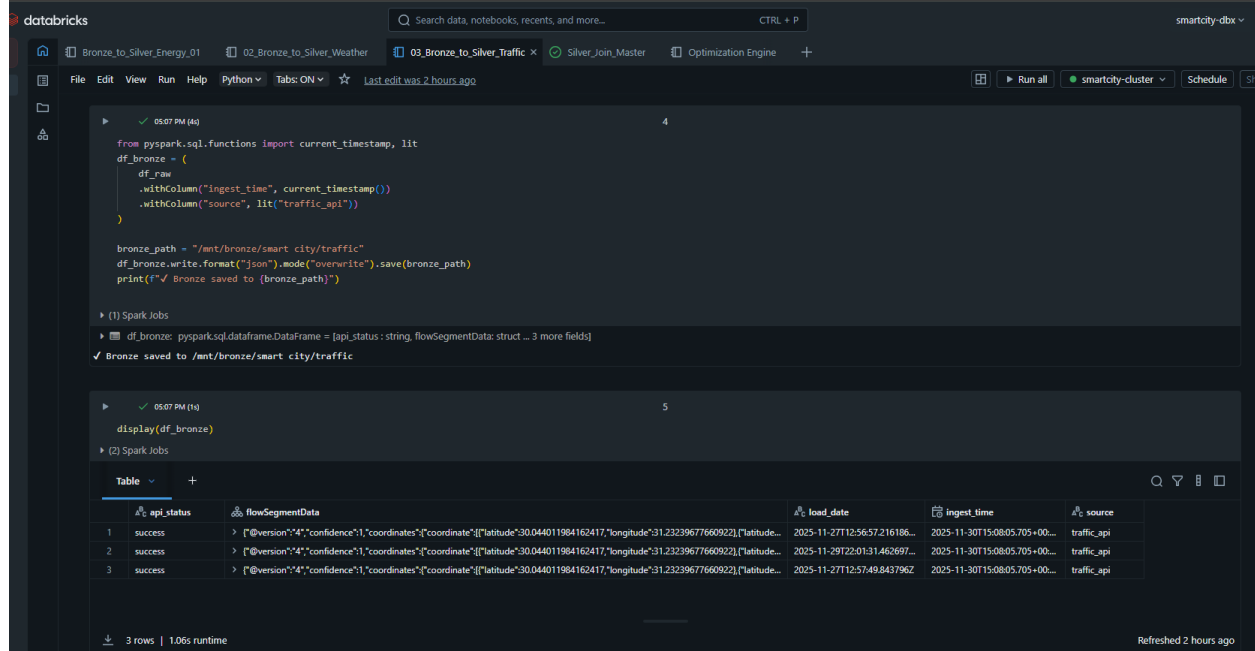
Output (3) Spark Jobs:

```
df_raw: pyspark.sql.dataframe.DataFrame = [api_status: string, flowSegmentData: struct ... 1 more field]
```

✓ RAW Loaded

	api_status	flowSegmentData	load_date
1	success	[{"@version": "4", "confidence": 1, "coordinates": {"coordinate": [{"latitude": 30.044011984162417, "longitude": 31.23239677660922}], "latitude...}	2025-11-27T12:56:57.216186...
2	success	[{"@version": "4", "confidence": 1, "coordinates": {"coordinate": [{"latitude": 30.044011984162417, "longitude": 31.23239677660922}], "latitude...}	2025-11-29T22:01:31.462697...

2 rows | 11.20s runtime



The screenshot shows a Databricks notebook with the following code and output:

```
from pyspark.sql.functions import current_timestamp, lit
df_bronze = (
    df_raw
      .withColumn("ingest_time", current_timestamp())
      .withColumn("source", lit("traffic_api"))
)

bronze_path = "/mnt/bronze/smart-city/traffic"
df_bronze.write.format("json").mode("overwrite").save(bronze_path)
print("✓ Bronze saved to {bronze_path}")
```

Output (1) Spark Jobs:

```
df_bronze: pyspark.sql.dataframe.DataFrame = [api_status: string, flowSegmentData: struct ... 3 more fields]
```

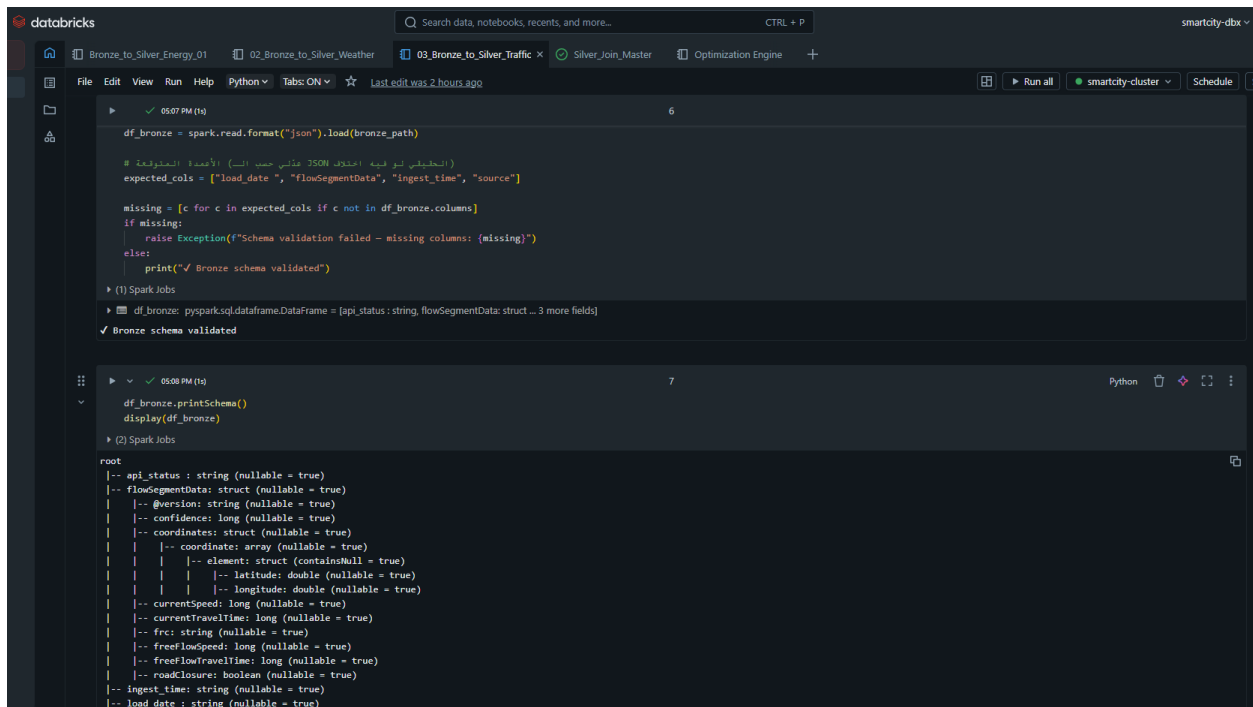
✓ Bronze saved to /mnt/bronze/smart-city/traffic

```
display(df_bronze)
```

Output (2) Spark Jobs:

	api_status	flowSegmentData	load_date	ingest_time	source
1	success	[{"@version": "4", "confidence": 1, "coordinates": {"coordinate": [{"latitude": 30.044011984162417, "longitude": 31.23239677660922}], "latitude...}	2025-11-27T12:56:57.216186...	2025-11-30T15:08:05.705+00...	traffic_api
2	success	[{"@version": "4", "confidence": 1, "coordinates": {"coordinate": [{"latitude": 30.044011984162417, "longitude": 31.23239677660922}], "latitude...}	2025-11-29T22:01:31.462697...	2025-11-30T15:08:05.705+00...	traffic_api
3	success	[{"@version": "4", "confidence": 1, "coordinates": {"coordinate": [{"latitude": 30.044011984162417, "longitude": 31.23239677660922}], "latitude...}	2025-11-27T12:57:49.843796Z	2025-11-30T15:08:05.705+00...	traffic_api

3 rows | 1.06s runtime



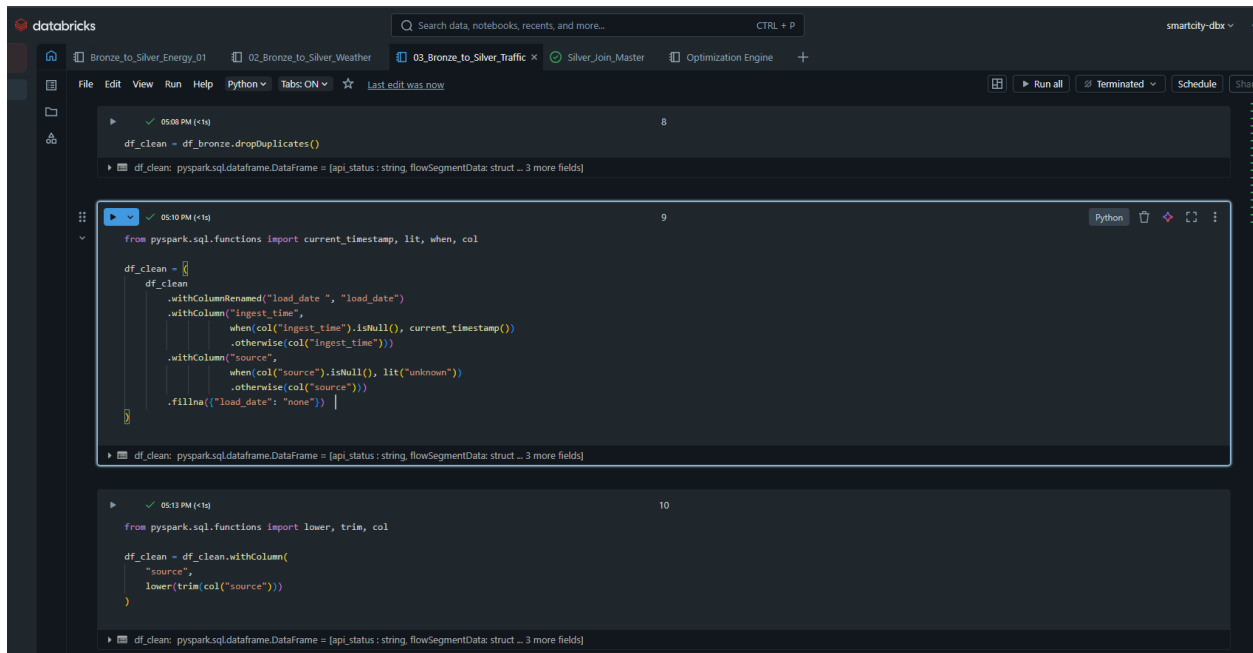
The screenshot shows a Databricks notebook with two code cells. The first cell, executed at 05:07 PM, loads a JSON file from a bronze path and validates its schema against expected columns: 'load_date', 'flowSegmentData', 'ingest_time', and 'source'. It raises an exception if any columns are missing. The second cell, executed at 05:08 PM, prints the schema of the resulting DataFrame. The schema is as follows:

```
root
|-- api_status : string (nullable = true)
|-- flowSegmentData: struct (nullable = true)
|   |-- @version: string (nullable = true)
|   |-- confidence: long (nullable = true)
|   |-- coordinates: struct (nullable = true)
|   |   |-- coordinate: array (nullable = true)
|   |   |   |-- element: struct (containsNull = true)
|   |   |   |-- latitude: double (nullable = true)
|   |   |   |-- longitude: double (nullable = true)
|   |-- currentSpeed: long (nullable = true)
|   |-- currentTravelTime: long (nullable = true)
|   |-- free: string (nullable = true)
|   |-- freeFlowSpeed: long (nullable = true)
|   |-- freeFlowTravelTime: long (nullable = true)
|   |-- roadClosure: boolean (nullable = true)
|   |-- ingest_time: string (nullable = true)
|   |-- load_date: string (nullable = true)
```

3. Data Cleaning Logic:

Remove Duplicates & Fix timestamp format + check hourly struct

Standardize strings & Missing Values



The screenshot shows a Databricks notebook with three code cells. The first cell, executed at 05:08 PM, drops duplicates from the DataFrame. The second cell, executed at 05:10 PM, imports functions from pyspark.sql.functions and applies transformations to the DataFrame: renaming 'load_date' to 'load_date', updating 'ingest_time' with 'current_timestamp()' if it's null, updating 'source' with 'lit("unknown")' if it's null, and filling 'load_date' with 'none' if it's null. The third cell, executed at 05:13 PM, applies 'lower' and 'trim' functions to the 'source' column.

```
df_clean = df_bronze.dropDuplicates()

df_clean = df_clean \
    .withColumnRenamed("load_date", "load_date") \
    .withColumn("ingest_time", \
        when(col("ingest_time").isNull(), current_timestamp()) \
        .otherwise(col("ingest_time"))) \
    .withColumn("source", \
        when(col("source").isNull(), lit("unknown")) \
        .otherwise(col("source"))) \
    .fillna({"load_date": "none"})

df_clean = df_clean.withColumn(
    "source",
    lower(trim(col("source")))
)
```

databricks

Search data, notebooks, recents, and more... CTRL + P smartcity-dbx

Bronze_to_Silver_Energy_01 02_Bronze_to_Silver_Weather 03_Bronze_to_Silver_Traffic x Silver_Join_Master Optimization Engine +

File Edit View Run Help Python Tabs ON ☆ Last edit was now Run all Terminated Schedule

05:13 PM (+1) 11 Python

```
import pyspark.sql.functions as F
df_clean = df_clean.withColumn("source", F.lower(F.trim(F.col("source"))))
```

df_clean: pyspark.sql.DataFrame = [api_status: string, flowSegmentData: struct ... 3 more fields]

05:13 PM (p) 12 Python

```
df_clean = df_clean.filter(col("flowSegmentData").isNotNull())

print("Cleaning done")
display(df_clean.limit(3))
```

(2) Spark Jobs

df_clean: pyspark.sql.DataFrame = [api_status: string, flowSegmentData: struct ... 3 more fields]

✓ Cleaning done

Table	+	🔍	📄	📊
api_status	flowSegmentData	ingest_time	load_date	source
1 success	> [{"version":"4","confidence":1,"coordinates":{"coordinate":{"latitude":30.044011984162417,"longitude":31.23239677660922},"latitude":30.044011984162417,"longitude":31.23239677660922},"latitude":30.044011984162417,"longitude":31.23239677660922}]	2025-11-30T15:08:02.33...	2025-11-27T12:56:57.216186...	traffic_api
2 success	> [{"version":"4","confidence":1,"coordinates":{"coordinate":{"latitude":30.044011984162417,"longitude":31.23239677660922},"latitude":30.044011984162417,"longitude":31.23239677660922},"latitude":30.044011984162417,"longitude":31.23239677660922}]	2025-11-30T15:08:02.33...	2025-11-29T22:01:31.462697...	traffic_api
3 success	> [{"version":"4","confidence":1,"coordinates":{"coordinate":{"latitude":30.044011984162417,"longitude":31.23239677660922},"latitude":30.044011984162417,"longitude":31.23239677660922},"latitude":30.044011984162417,"longitude":31.23239677660922}]	2025-11-30T15:08:02.33...	2025-11-27T12:57:49.8437962	traffic_api

3 rows | 2.47s runtime Refreshed 2 hours ago

4. silver Transformation

databricks

Search data, notebooks, recents, and more... CTRL + P smartcity-dbx

Bronze_to_Silver_Energy_01 02_Bronze_to_Silver_Weather 03_Bronze_to_Silver_Traffic x Silver_Join_Master Optimization Engine +

File Edit View Run Help Python Tabs ON ☆ Last edit was 3 minutes ago Run all Terminated Schedule

05:17 PM (+1) 13 Python

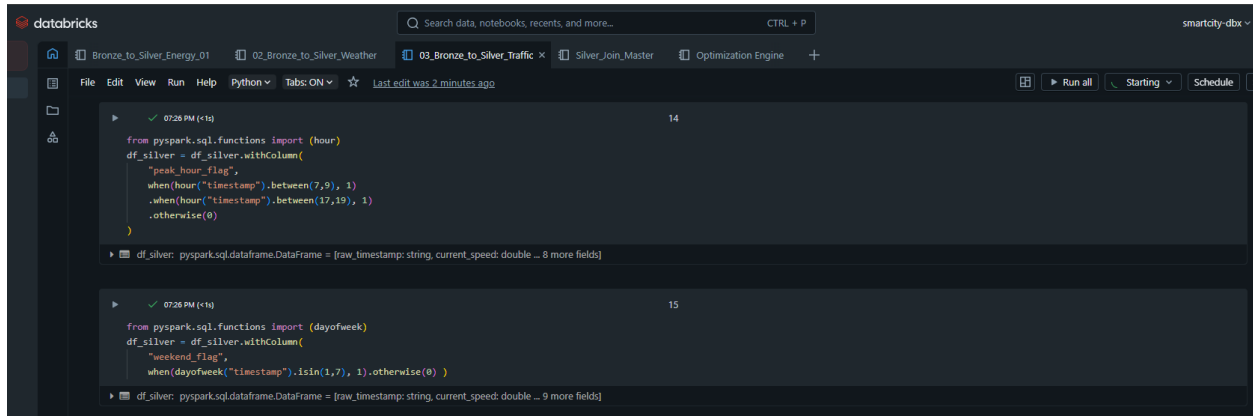
```
from pyspark.sql.functions import (
    col, when, lit, round, current_timestamp,
    to_timestamp, lower, trim
)

df_silver = (
    df_clean
    .select(
        col("load_date").alias("raw_timestamp"),
        col("flowSegmentData.currentSpeed").alias("current_speed"),
        col("flowSegmentData.freeFlowSpeed").alias("free_flow_speed"),
        col("ingest_time"),
        col("source")
    )
    .withColumn("timestamp", to_timestamp(col("raw_timestamp")))
    .withColumn("current_speed", col("current_speed").cast("double"))
    .withColumn("free_flow_speed", col("free_flow_speed").cast("double"))
    .withColumn(
        "congestion_index",
        when(
            (col("free_flow_speed").isNotNull()) & (col("free_flow_speed") != 0),
            round((col("free_flow_speed") - col("current_speed")) / col("free_flow_speed"), 2)
        ).otherwise(lit(None))
    )
    .withColumn(
        "traffic_status",
        when(col("congestion_index") >= 0.6, lit("Heavy"))
        .when(col("congestion_index") >= 0.3, lit("Moderate"))
        .when(col("congestion_index").isNotNull(), lit("Free Flow"))
        .otherwise(lit("Unknown"))
    )
    .withColumn("processing_time", current_timestamp())
)
```

df_silver: pyspark.sql.DataFrame = [raw_timestamp: string, current_speed: double ... 7 more fields]

5. FEATURE ENGINEERING :

peak_hour_flag & weekend_flag & congestion_index
then saved on silver



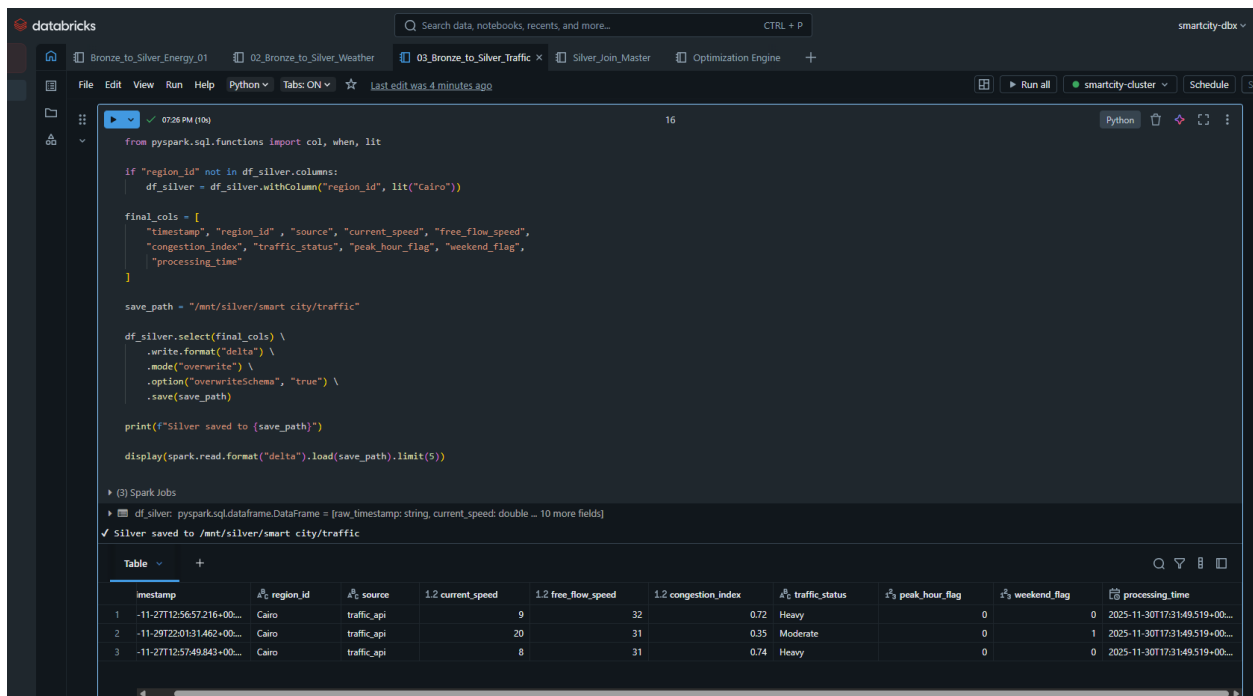
The screenshot shows a Databricks notebook with two code cells. The first cell adds a 'peak_hour_flag' column to the 'df_silver' DataFrame, setting it to 1 for timestamps between 7:00 and 19:00, and 0 otherwise. The second cell adds a 'weekend_flag' column, setting it to 1 for days of the week 1 (Monday) and 7 (Sunday), and 0 otherwise.

```
from pyspark.sql.functions import (hour)
df_silver = df_silver.withColumn(
    "peak_hour_flag",
    when(hour("timestamp").between(7,9), 1)
    .when(hour("timestamp").between(17,19), 1)
    .otherwise(0)
)

df_silver: pyspark.sql.dataframe.DataFrame = [raw_timestamp: string, current_speed: double ... 8 more fields]
```

```
from pyspark.sql.functions import (dayofweek)
df_silver = df_silver.withColumn(
    "weekend_flag",
    when(dayofweek("timestamp").isin(1,7), 1).otherwise(0)
)

df_silver: pyspark.sql.dataframe.DataFrame = [raw_timestamp: string, current_speed: double ... 9 more fields]
```



The screenshot shows a Databricks notebook with a code cell that saves the 'df_silver' DataFrame to a Delta table. The code selects specific columns, writes them to a path, and then displays the first five rows of the saved data. Below the code, a table preview is shown with columns: timestamp, region_id, source, current_speed, free_flow_speed, congestion_index, traffic_status, peak_hour_flag, weekend_flag, and processing_time.

```
from pyspark.sql.functions import col, when, lit

if "region_id" not in df_silver.columns:
    df_silver = df_silver.withColumn("region_id", lit("Cairo"))

final_cols = [
    "timestamp", "region_id", "source", "current_speed", "free_flow_speed",
    "congestion_index", "traffic_status", "peak_hour_flag", "weekend_flag",
    "processing_time"
]

save_path = "/mnt/silver/smart city/traffic"

df_silver.select(final_cols) \
    .write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(save_path)

print(f"Silver saved to {save_path}")

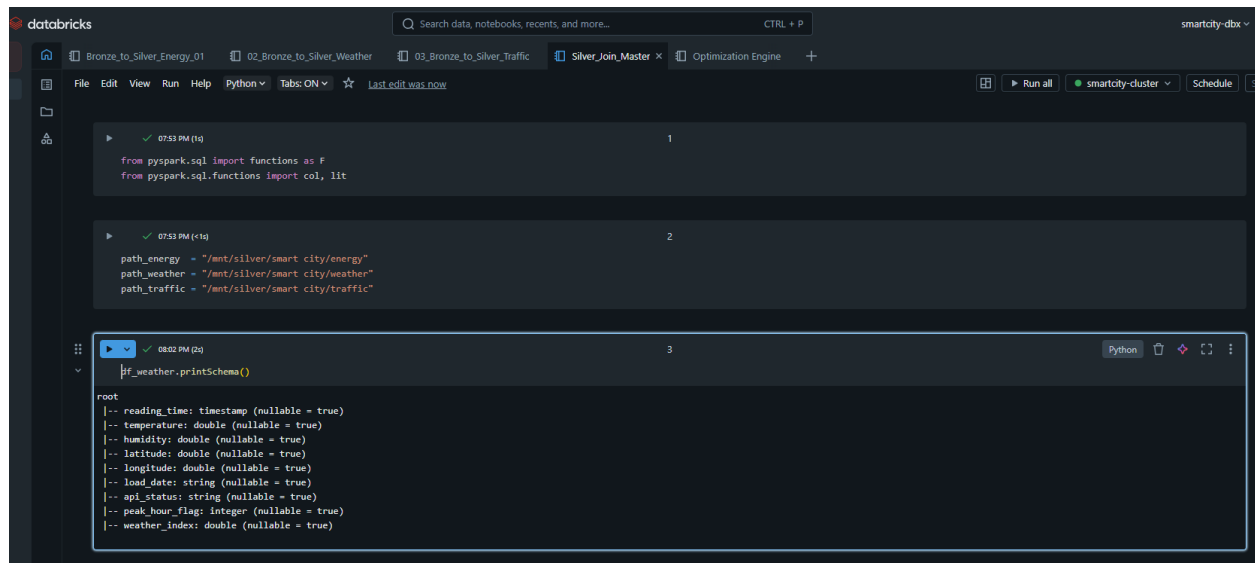
display(spark.read.format("delta").load(save_path).limit(5))

(3) Spark Jobs
df_silver: pyspark.sql.dataframe.DataFrame = [raw_timestamp: string, current_speed: double ... 10 more fields]
✓ Silver saved to /mnt/silver/smart city/traffic
```

	timestamp	region_id	source	current_speed	free_flow_speed	congestion_index	traffic_status	peak_hour_flag	weekend_flag	processing_time
1	-11-27T12:56:57.216+00...	Cairo	traffic_api	9	32	0.72	Heavy	0	0	2025-11-30T17:31:49.519+00...
2	-11-29T22:01:31.462+00...	Cairo	traffic_api	20	31	0.35	Moderate	0	1	2025-11-30T17:31:49.519+00...
3	-11-27T12:57:49.843+00...	Cairo	traffic_api	8	31	0.74	Heavy	0	0	2025-11-30T17:31:49.519+00...

4. Silver_Join_Master

Join energy + weather + traffic → Build final Master Table in delta formate



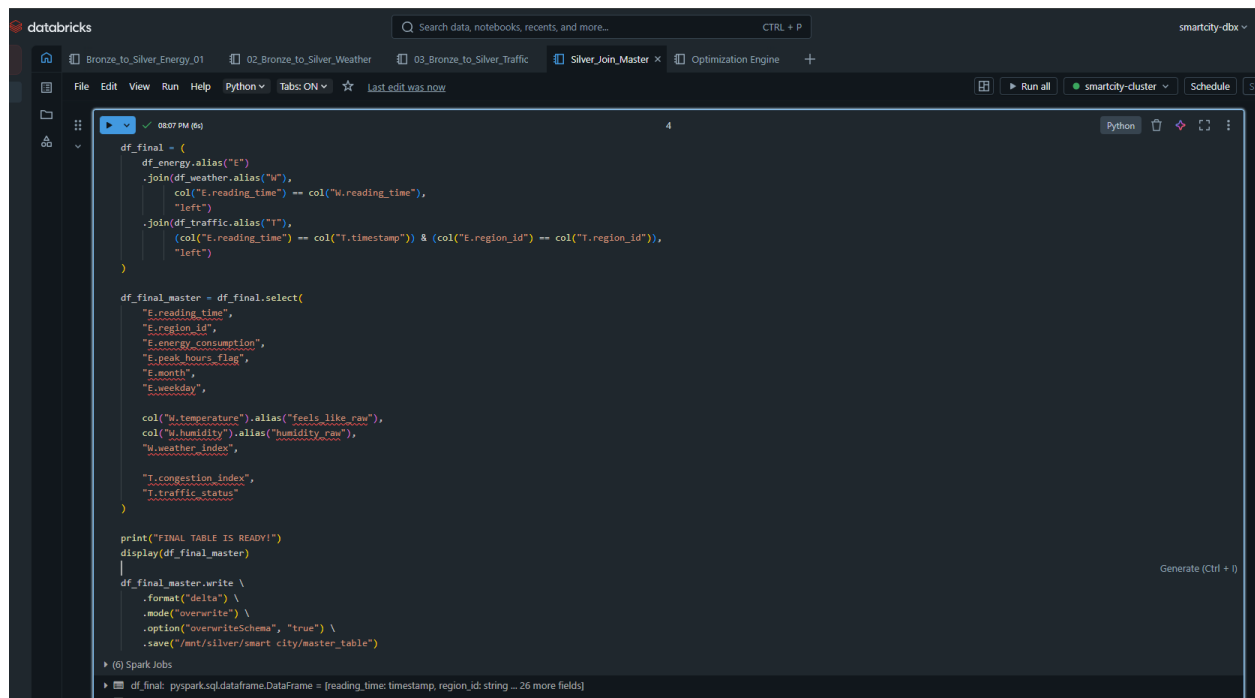
The screenshot shows a Databricks notebook with three steps:

```
from pyspark.sql import functions as F
from pyspark.sql.functions import col, lit
```

```
path_energy = "/mnt/silver/smart city/energy"
path_weather = "/mnt/silver/smart city/weather"
path_traffic = "/mnt/silver/smart city/traffic"
```

```
df_weather.printSchema()
```

```
root
 |-- reading_time: timestamp (nullable = true)
 |-- temperature: double (nullable = true)
 |-- humidity: double (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- load_date: string (nullable = true)
 |-- api_status: string (nullable = true)
 |-- peak_hour_flag: integer (nullable = true)
 |-- weather_index: double (nullable = true)
```



The screenshot shows the fourth step of the notebook, which performs a join and writes the final master table in Delta format:

```
df_final = (
    df_energy.alias("E")
        .join(df_weather.alias("W"),
              col("E.reading_time") == col("W.reading_time"),
              "left")
        .join(df_traffic.alias("T"),
              (col("E.reading_time") == col("T.timestamp")) & (col("E.region_id") == col("T.region_id")),
              "left")
)

df_final_master = df_final.select(
    "E.reading_time",
    "E.region_id",
    "E.energy_consumption",
    "E.peak_hours_flag",
    "E.month",
    "E.weekday",
    col("W.temperature").alias("feels_like_raw"),
    col("W.humidity").alias("humidity_raw"),
    "W.weather_index",
    "T.congestion_index",
    "T.traffic_status"
)

print("FINAL TABLE IS READY!")
display(df_final_master)

df_final_master.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save("/mnt/silver/smart city/master_table")
```

Below the code, the Spark Jobs section shows the execution of the final step:

```
df_final: pyspark.sql.dataframe.DataFrame = [reading_time: timestamp, region_id: string ... 26 more fields]
df_final_master: pyspark.sql.dataframe.DataFrame = [reading_time: timestamp, region_id: string ... 9 more fields]
```

remove null from the join then save it in the silver container

The screenshot shows a Databricks workspace with a notebook titled 'Silver_Join_Master'. The notebook contains Python code that calculates averages for 'feels_like_raw', 'humidity_raw', and 'congestion_index' from a DataFrame 'df_final_master'. It then fills nulls in these columns using the 'fillna()' method with default values of 0. The code also prints a confirmation message and displays the resulting DataFrame.

```
import pyspark.sql.functions as F

# 1. Calculate averages using the CORRECT column names (_raw)
avg_feels = df_final_master.agg(F.avg("feels_like_raw")).first()[0]
avg_humid = df_final_master.agg(F.avg("humidity_raw")).first()[0]
avg_congest = df_final_master.agg(F.avg("congestion_index")).first()[0]

# 2. Fill Nulls using the same (_raw) names
final_master_table = (
    df_final_master
    .fillna({
        "feels_like_raw": avg_feels if avg_feels else 0,
        "humidity_raw": avg_humid if avg_humid else 0,
        "congestion_index": avg_congest if avg_congest else 0,
        "traffic_status": "Unknown"
    })
)

print("Nulls filled successfully.")
display(final_master_table)
```

The output shows a table with 13 rows and 11 columns. The columns are: reading_time, region_id, energy_consumption, peak_hours_flag, month, weekday, feels_like_raw, humidity_raw, weather_index, and congestion_index. The data is as follows:

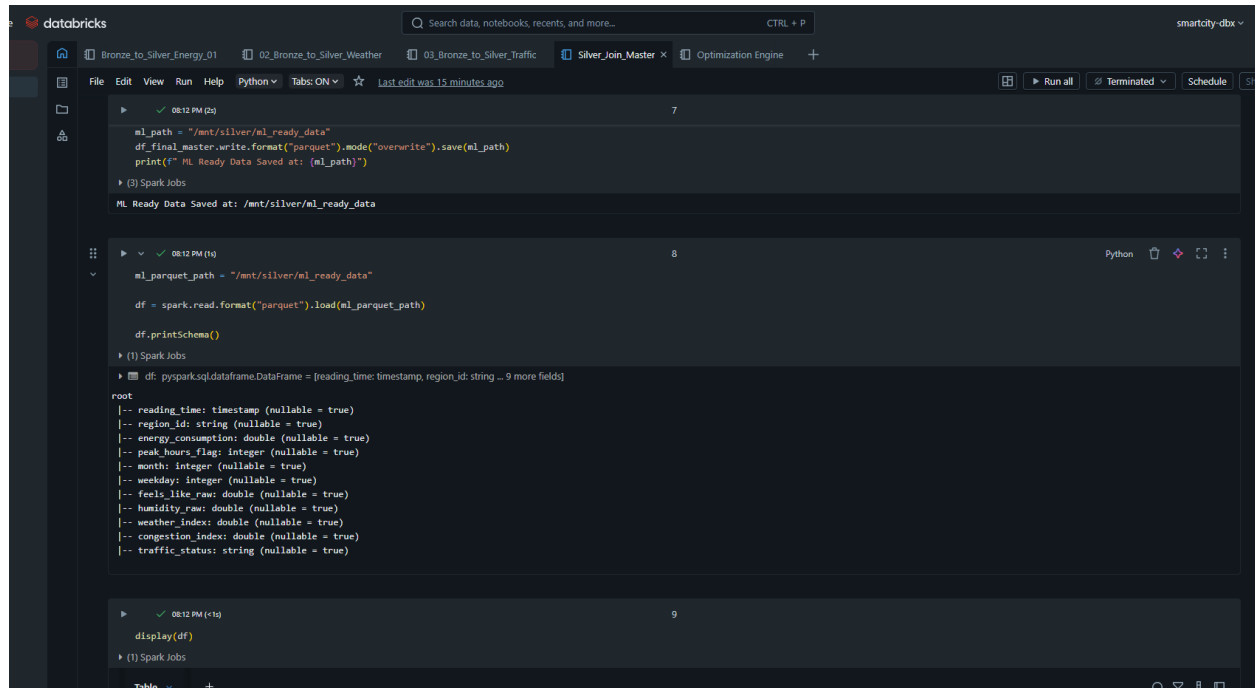
	reading_time	region_id	energy_consumption	peak_hours_flag	month	weekday	feels_like_raw	humidity_raw	weather_index	congestion_index
6	2025-12-05T21:00:00.000+00...	r2	129.1763524431427	1	12	6	13.4	74	31.58	
7	2025-12-02T00:00:00.000+00...	r2	332.3918364745318	0	12	3	14.1	94	38.07	
8	2025-12-03T10:00:00.000+00...	r3	163.09665571073538	0	12	4	21.8	56	32.06	
9	2025-12-03T00:00:00.000+00...	r2	127.69553382794757	0	12	4	14.5	93	38.05	
10	2025-12-05T15:00:00.000+00...	r3	278.4211731808441	0	12	6	20.7	47	28.589999999999996	
11	2025-12-03T07:00:00.000+00...	r2	110.6409965551321	0	12	4	16.3	81	35.71	
12	2025-12-05T04:00:00.000+00...	r1	207.02304813876097	0	12	6	13.6	71	30.82	
13	2025-12-03T02:00:00.000+00...	r3	266.320204944712	0	12	4	13.8	94	37.86	

The second screenshot shows the same notebook with a new cell added, which writes the DataFrame to a Delta table. The code is:

```
df_final_master.write.format("delta").mode("overwrite").save("/mnt/silver/smart-city/master_table")
print("Master Table Saved (Delta)")
```

The output shows the message 'Master Table Saved (Delta)'.

then transform the master table from delta to parquet for the ML (machine learning) in gold layer



The screenshot shows a Databricks notebook interface with the following components:

- Top Bar:** Databricks logo, search bar, and tabs for "Bronze_To_Silver_Energy_01", "D2_Bronze_To_Silver_Weather", "D3_Bronze_To_Silver_Traffic", "Silver_Join_Master" (active), and "Optimization Engine".
- File Menu:** File, Edit, View, Run, Help, Python, Tabs: ON, Last edit was 15 minutes ago.
- Code Editor:**
 - Cell 7:** Saves the master table to Parquet format.

```
ml_path = "/mnt/silver/ml_ready_data"
df_final_master.write.format("parquet").mode("overwrite").save(ml_path)
print(f" ML Ready Data Saved at: {ml_path}")
```
 - Cell 8:** Loads the Parquet data and prints the schema.

```
ml_parquet_path = "/mnt/silver/ml_ready_data"
df = spark.read.format("parquet").load(ml_parquet_path)
df.printSchema()
```
 - Cell 9:** Displays the DataFrame.

```
display(df)
```
- Output:**
 - Cell 7:** "ML Ready Data Saved at: /mnt/silver/ml_ready_data"
 - Cell 8:** Schema output:

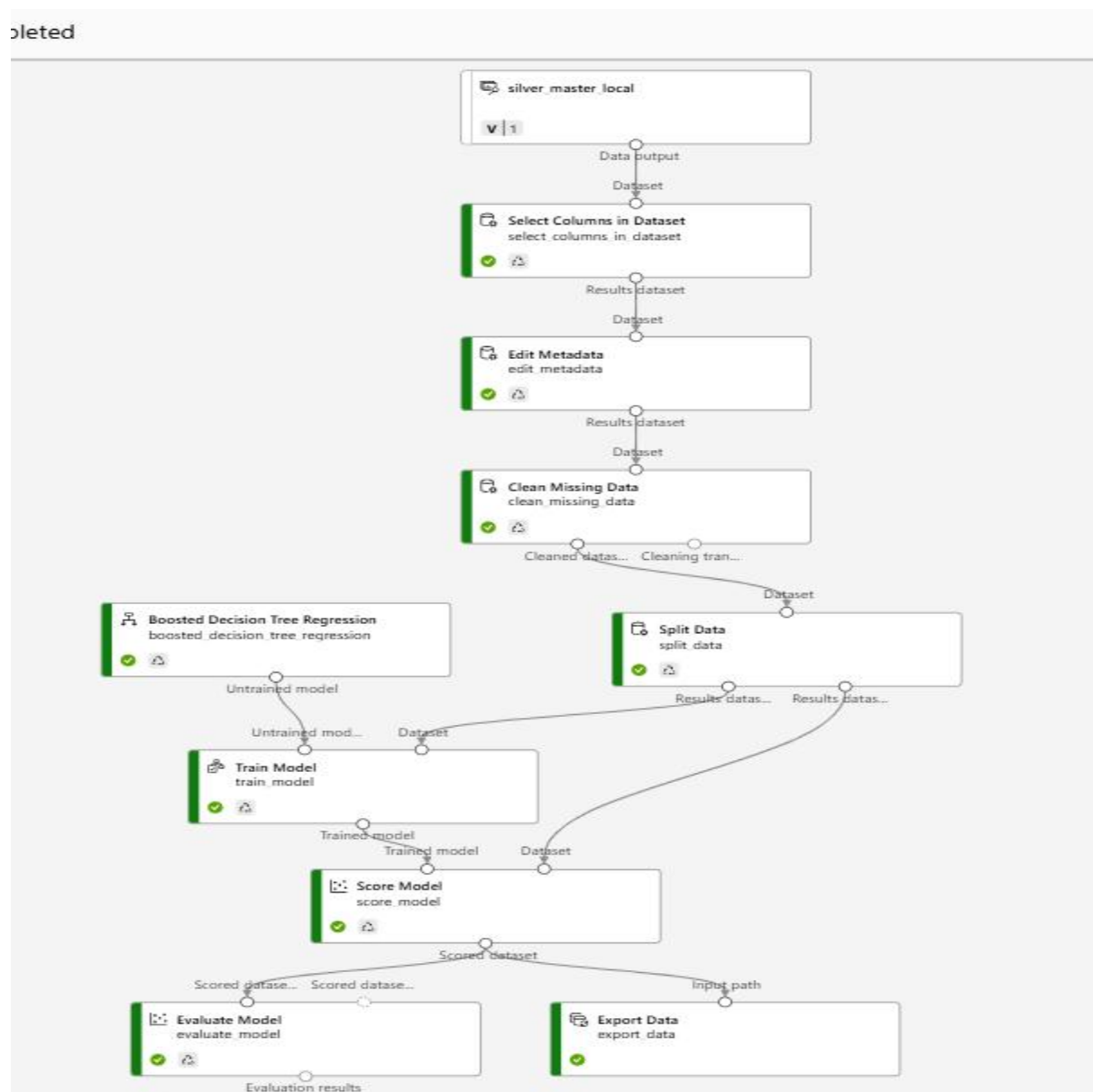
```
root
 |-- reading_time: timestamp (nullable = true)
 |-- region_id: string (nullable = true)
 |-- energy_consumption: double (nullable = true)
 |-- peak_hours_flag: integer (nullable = true)
 |-- month: integer (nullable = true)
 |-- weekday: integer (nullable = true)
 |-- feels_like_raw: double (nullable = true)
 |-- humidity_raw: double (nullable = true)
 |-- weather_index: double (nullable = true)
 |-- congestion_index: double (nullable = true)
 |-- traffic_status: string (nullable = true)
```
 - Cell 9:** A table view of the data.

5. Gold Layer (Machine Learning)

In this layer, refined data is utilized to generate predictive insights and actionable recommendations using Azure Machine Learning.

5.1 Azure ML Designer Pipeline

Pipeline Overview: The pipeline is designed to ingest time-series data from the Silver layer, process it, and train a forecasting model to predict future energy consumption.



Blocks Used The pipeline follows a structured data flow consisting of the following modules:

- 1. Input Data (silver_master_local):** Ingests the processed data from the Silver layer.
- 2. Select Columns in Dataset:** Selects relevant features for training and excludes unnecessary columns (e.g., IDs not needed for prediction).
- 3. Edit Metadata:** Defines column roles (marking the "Label" or target column) and ensures correct data types (Categorical vs. Numeric).
- 4. Clean Missing Data:** Handles null values to prevent model errors (likely using substitution or removal).
- 5. Split Data:** Divides the dataset into Training and Testing sets to validate performance.
- 6. Boosted Decision Tree Regression:** The specific machine learning algorithm selected for the regression task.
- 7. Train Model:** Trains the algorithm using the training dataset.
- 8. Score Model:** Generates predictions on the test dataset.
- 9. Evaluate Model:** Compares predictions vs. actuals to calculate error metrics.
- 10. Export Data:** Writes the final predictions (Gold data) back to the storage/database.

Data Processing

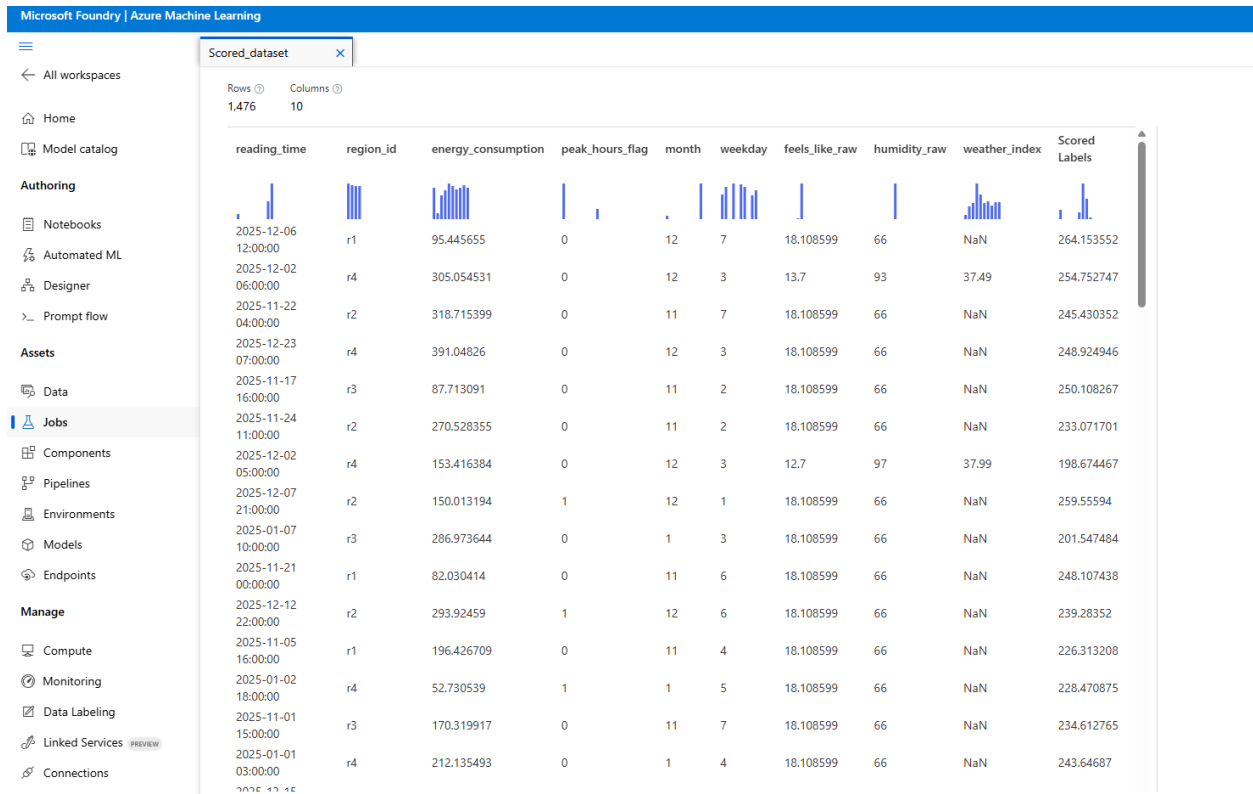
- **Feature Selection:** The pipeline explicitly filters input columns to focus on high-impact features.
- **Metadata Management:** The Edit Metadata block is utilized to explicitly cast data types and identify the target variable (Energy Consumption).
- **Imputation:** Missing values are systematically handled via the Clean Missing Data module before the data enters the training phase.

Model Used

- **Algorithm: Boosted Decision Tree Regression.**
- **Reasoning:**
 - This algorithm was chosen for its high accuracy and efficiency in regression tasks.
 - It uses an ensemble of decision trees (Gradient Boosting) to correct errors from previous trees, making it robust against non-linear patterns in energy consumption data.
 - It handles tabular data effectively and offers good interpretability.

5.2 Forecast Results

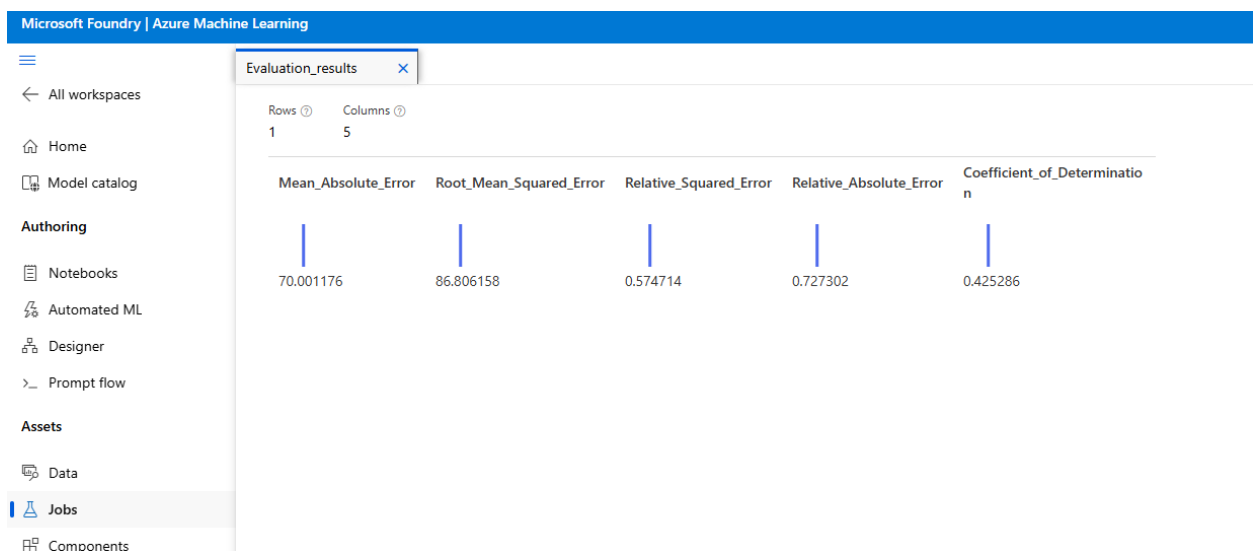
Predicted vs Actual: The model demonstrates a strong ability to follow the trend of energy consumption. The visualization below shows the alignment between the Actual Load and Predicted Load generated from the Score Model module.



Error Metrics:

The regression performance was evaluated using standard metrics from the **Evaluate Model** module:

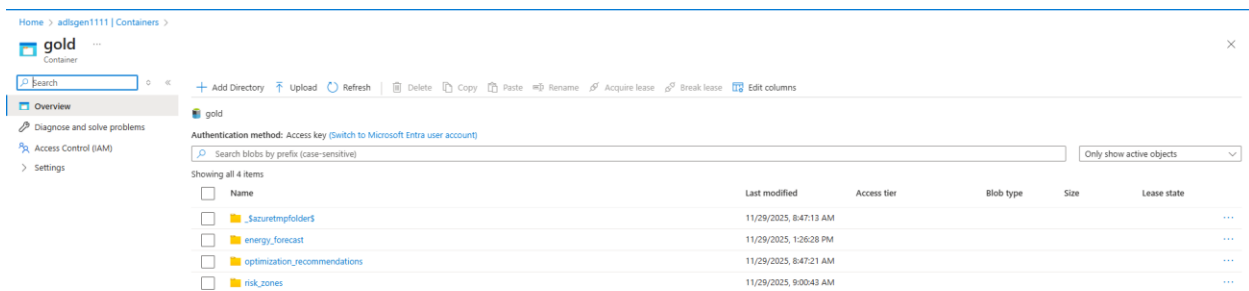
- **Coefficient of Determination (R^2):** Indicates how well the model explains the variance in the data. A value closer to 1 indicates a high fit.
- **RMSE (Root Mean Squared Error):** Represents the standard deviation of the prediction errors.
- **MAE (Mean Absolute Error):** The average magnitude of errors in the predictions.



5.3 Gold Tables

The output of the ML model and the optimization logic is stored in the following Gold Delta Tables:

- 1. gold.energy_forecast** *Stores the predicted energy consumption for the next 7 days.*
- 2. gold.risk_zones** *Identifies areas predicted to exceed capacity thresholds.*
- 3. gold.optimization_recommendations** *Actionable steps generated by the engine.*



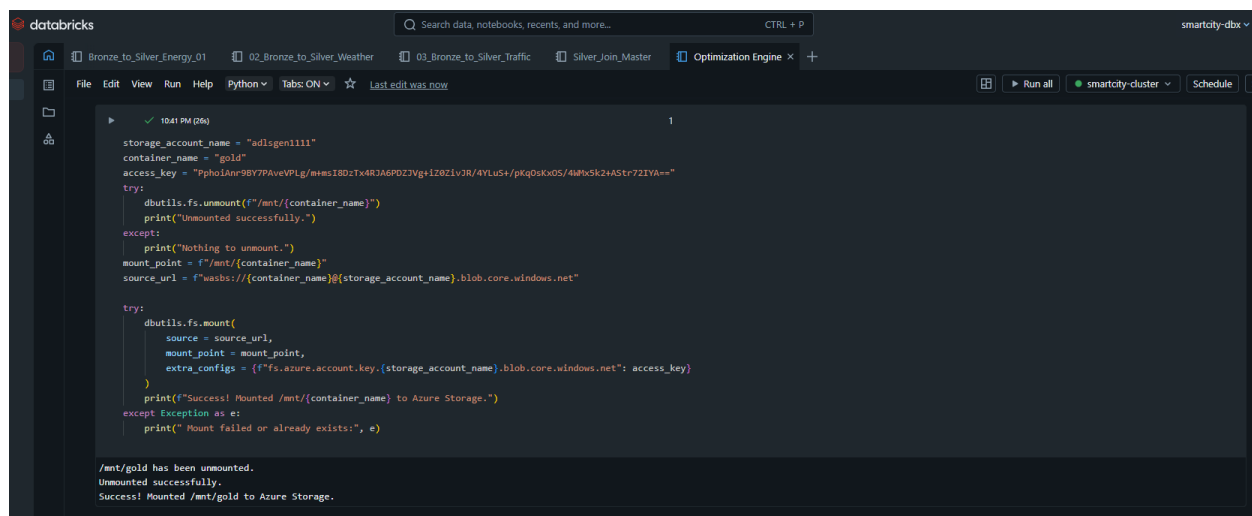
Name	Last modified	Access tier	Blob type	Size	Lease state
_\$azuremgmfolders\$	11/29/2025, 8:47:13 AM				...
energy_forecast	11/29/2025, 1:26:28 PM				...
optimization_recommendations	11/29/2025, 8:47:21 AM				...
risk_zones	11/29/2025, 9:00:43 AM				...



5.4 Optimization Engine Logic

This module uses the forecast data to generate smart recommendations:

- **Peak Shifting:** Identify hours where predicted load > 90% of grid capacity and suggest shifting non-critical loads (e.g., industrial processes) to off-peak hours (12 AM - 6 AM).
- **Load Balancing:** Detect imbalance between regions. If Region A is overloaded and Region B is underutilized, suggest rerouting power distribution if grid topology permits.
- **Critical Regions:** Flag any region where predicted_load exceeds the safety threshold for more than 2 consecutive hours as a "Critical Zone" requiring immediate attention.



The screenshot shows a Databricks notebook interface with a dark theme. The top bar includes the Databricks logo, a search bar, and a 'CTRL + P' shortcut. The notebook has several tabs open: 'Bronze_to_Silver_Energy_01', '02_Bronze_to_Silver_Weather', '03_Bronze_to_Silver_Traffic', 'Silver_Join_Master', and 'Optimization Engine'. The 'Optimization Engine' tab is active. The code editor shows a Python script that attempts to unmount and then mount an Azure Storage container. The script uses 'dbutils.fs' for file system operations. The output at the bottom shows the execution results: '/mnt/gold has been unmounted. Unmounted successfully. Success! Mounted /mnt/gold to Azure Storage.'

```
storage_account_name = "adlsgen1111"
container_name = "gold"
access_key = "PphoiAnr9BV7PAveVPLg/ehes18Dz7x4RJA6PD2JVg+120Z1v3R/4YLU5+/pKq0skX0S/4WpW5K2+ASTr721YA=="

try:
    dbutils.fs.unmount(f"/mnt/{container_name}")
    print("Unmounted successfully.")
except:
    print("Nothing to unmount.")
mount_point = f"/mnt/{container_name}"
source_url = f"wasbs://{container_name}@{storage_account_name}.blob.core.windows.net"

try:
    dbutils.fs.mount(
        source = source_url,
        mount_point = mount_point,
        extra_configs = {f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net": access_key}
    )
    print(f"Success! Mounted /mnt/{container_name} to Azure Storage.")
except Exception as e:
    print("Mount failed or already exists:", e)
```

/mnt/gold has been unmounted.
Unmounted successfully.
Success! Mounted /mnt/gold to Azure Storage.

databricks

Search data, notebooks, recents, and more...

CTRL + P

smartcity-dbx

Bronze_to_Silver_Energy_01

02_Bronze_to_Silver_Weather

03_Bronze_to_Silver_Traffic

Silver_Join_Master

Optimization Engine

File Edit View Run Help Python Tabs ON Last edit was 1 minute ago

Run all smartcity-cluster Schedule

2 minutes ago (1s)

4

display(dbutils.fs.mounts())

(2) Spark Jobs

Table	mountPoint	source	encryptionType
1	/databricks-datasets	databricks-datasets	
2	/mnt/gold	wasbs://gold@adlsgen1111.blob.core.windows.net	
3	/Volumes	UnityCatalogVolumes	
4	/mnt/silver	wasbs://silver@adlsgen1111.blob.core.windows.net/	
5	/databricks/milflow-track...	databricks/milflow-tracking	
6	/databricks-results	databricks-results	
7	/mnt/raw	wasbs://raw@adlsgen1111.blob.core.windows.net/	
8	/databricks/milflow-registry	databricks/milflow-registry	
9	/mnt/bronze	wasbs://bronze@adlsgen1111.blob.core.windows.n...	
10	/Volume	DbfsReserved	
11	/volumes	DbfsReserved	
12	/	DatabricksRoot	
13	/volume	DbfsReserved	

13 rows | 0.79s runtime

Refreshed 1 minute ago

2 minutes ago (1s)

5

display(dbutils.fs.ls("/mnt/gold/"))

(2) Spark Jobs

Table	path	name	size	modificationTime
1	dbfs/mnt/gold/energy_forecast/	energy_forecast/	0	0

databricks

Search data, notebooks, recents, and more...

CTRL + P

smartcity-dbx

Bronze_to_Silver_Energy_01

02_Bronze_to_Silver_Weather

03_Bronze_to_Silver_Traffic

Silver_Join_Master

Optimization Engine

File Edit View Run Help Python Tabs ON Last edit was 2 minutes ago

Run all smartcity-cluster Schedule

2 minutes ago (2s)

6

```
from pyspark.sql.functions import col, when

df_forecast = spark.read.format("delta").load("/mnt/gold/energy_forecast")

peak_threshold = 2000

df_risk = df_forecast.withColumn(
    "risk_label",
    when(col("predicted_consumption") > peak_threshold, "High Risk")
    .when(col("predicted_consumption") > (peak_threshold * 0.8), "Medium Risk")
    .otherwise("Low Risk")
)

df_risk.write.format("delta").mode("overwrite").option("mergeSchema", "true").save("/mnt/gold/risk_zones")

print(" Gold Table Created: risk_zones")
display(df_risk.limit(5))
```

(2) Spark Jobs

df_forecast: pyspark.sql.dataframe.DataFrame = [reading_time: string, region_id: string ... 10 more fields]

df_risk: pyspark.sql.dataframe.DataFrame = [reading_time: string, region_id: string ... 11 more fields]

Gold Table Created: risk_zones

Table	reading_time	region_id	energy_consumption	peak_hours_flag	month	weekday	feels_like_raw	humidity_raw	weather_index	scored_labels	predict
1	2025-12-06 12:00:00	r1	95.44565467551894	0	12	7	18.108598726114653	66	null	264.15355159216114	
2	2025-12-02 06:00:00	r4	305.05453141473254	0	12	3	13.7	93	37.489999999999995	254.75274746174793	
3	2025-11-22 04:00:00	r2	318.7153993362996	0	11	7	18.108598726114653	66	null	245.43035205516907	
4	2025-12-23 07:00:00	r4	391.0482600549623	0	12	3	18.108598726114653	66	null	248.9249456284177	
5	2025-11-17 16:00:00	r3	87.7130906353044	0	11	2	18.108598726114653	66	null	250.10826734954102	

5 rows | 1.09s runtime

Refreshed 1 minute ago

[Shift+Enter] to run and move to next cell

6. Synapse & SQL Views

Azure Synapse Analytics Serverless SQL Pool is used to serve the data to BI tools (Power BI) via T-SQL Views.

Views

- **view_energy_forecast:** Aggregates forecast data for reporting dashboard.
- **view_risk_zones:** Filters only high-risk zones for the alert system.
- **view_optimization_recommendations:** Joins recommendations with region metadata for the operations team.

The screenshot displays the Azure Synapse Analytics SQL editor interface. At the top, a warning message states: "Other users in your workspace may have access to modify this item. Do not use this item unless you trust all users who may have access to the workspace." Below this, the editor shows a T-SQL script for creating a database and a view. The script includes a check for the existence of the 'smart_city_db' database, its creation if it doesn't exist, and the creation of the 'view_energy_forecast' view. The view selects data from an OPENROWSET function that fetches data from a URL. Below the script, the 'Results' tab is active, showing a table with 10 columns: reading_time, region_id, energy_consu..., peak_hours_flag, month, weekday, feels_like_raw, humidity_raw, weather_index, and scored_labels. The table contains 5 rows of data. At the bottom, a status bar indicates "00:00:22 Query executed successfully."

```
1 IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'smart_city_db')
2 BEGIN
3     CREATE DATABASE smart_city_db;
4 END
5
6 USE smart_city_db;
7 GO
8
9 -----
10 -- 1. View for Energy Forecast
11 -----
12 CREATE OR ALTER VIEW view_energy_forecast AS
13 SELECT *
14 FROM OPENROWSET(
15     BULK 'https://adlsgen1111.dfs.core.windows.net/gold/energy_forecast',
16     FORMAT = 'DELTA'
17 ) AS [rows];
18 GO
19
20 -----
```

reading_time	region_id	energy_consu...	peak_hours_flag	month	weekday	feels_like_raw	humidity_raw	weather_index	scored_labels
2025-12-06 12:...	r1	95.4456546755...	0	12	7	18.1085987261...	66	(NULL)	264.153551592...
2025-12-02 06:...	r4	305.054531414...	0	12	3	13.7	93	37.4899999999...	254.752747461...
2025-11-22 04:...	r2	318.715399336...	0	11	7	18.1085987261...	66	(NULL)	245.430352055...
2025-12-23 07:...	r4	391.048260054...	0	12	3	18.1085987261...	66	(NULL)	248.924945628...
2025-11-17 16:...	r3	87.7130906635...	0	11	2	18.1085987261...	66	(NULL)	250.108267349...

00:00:22 Query executed successfully.

SQL script 1

Other users in your workspace may have access to modify this item. Do not use this item unless you trust all users who may have access to the workspace.

Run Undo Publish Query plan Connect to Built-in Use database smart_city_db

```

20
21 -- 2. View for Risk Zones
22
23 CREATE OR ALTER VIEW view_risk_zones AS
24 SELECT *
25 FROM OPENROWSET(
26     BULK 'https://adlsgen1111.dfs.core.windows.net/gold/risk_zones',
27     FORMAT = 'DELTA'
28 ) AS [rows];
29 GO
30
31
32 -- 3. View for Optimization Recommendations
33
34 CREATE OR ALTER VIEW view_optimization_recommendations AS
35 SELECT *
36 FROM OPENROWSET(
37     BULK 'https://adlsgen1111.dfs.core.windows.net/gold/optimization_recommendations',
38     FORMAT = 'DELTA'
39 ) AS [rows];
40

```

Results Messages

View Table Chart Export results

Search

reading_time	region_id	energy_consu...	peak_hours_flag	month	weekday	feels_like_raw	humidity_raw	weather_index	scored_labels
2025-12-06 12:...	r1	95.4456546755...	0	12	7	18.1085987261...	66	(NULL)	264.153551592...
2025-12-02 06:...	r4	305.054531414...	0	12	3	13.7	93	37.4899999999...	254.752747461...
2025-11-22 04:...	r2	318.715399336...	0	11	7	18.1085987261...	66	(NULL)	245.430352055...
2025-12-23 07:...	r4	391.048260054...	0	12	3	18.1085987261...	66	(NULL)	248.924945628...
2025-11-17 16:...	r3	87.7130906635...	0	11	2	18.1085987261...	66	(NULL)	250.108267349...

00:00:22 Query executed successfully.

7. Power BI Dashboard

The Power BI dashboard serves as the interactive visualization layer, translating complex data and ML predictions into actionable insights for grid operators and decision-makers.

7.1 Dashboard Pages

The report is divided into thematic pages to address different analytical needs:

- **Energy Overview:** A high-level executive summary displaying real-time consumption trends, historical comparisons, and aggregate usage across all regions.

- **Weather Impact:** Visualizes the correlation between weather conditions (Temperature, Humidity) and energy demand, highlighting how environmental factors drive load spikes.
- **Traffic Correlation:** Analyzes the relationship between city mobility/traffic density and energy consumption, proving the hypothesis of urban activity impact.
- **Forecasting:** Dedicated view for the Machine Learning results, plotting the "Actual vs. Predicted" load for the upcoming week with confidence intervals.
- **Risk Zones Heatmap:** A geospatial map (filled map) that color-codes regions based on risk levels (High/Medium/Low), allowing operators to pinpoint critical zones instantly.
- **Recommendations Panel:** A tabular view presenting the optimization engine's suggestions (e.g., "Shift Load in Zone A") with estimated cost savings.

7.2 KPIs Used (Key Performance Indicators)

To monitor grid health, the following metrics were defined using DAX:

- **Total Energy Consumption (kWh):** The sum of energy usage over the selected period.
- **Peak Load (kW):** The maximum recorded power demand, crucial for capacity planning.
- **Forecast Accuracy (%):** calculated dynamically ($1 - \text{MAPE}$) to track the reliability of the ML model.
- **Risk Index:** A count of regions currently flagged as "High Risk".

Smart City Energy Optimization & Risk Dashboard



Total Energy Consumption

302.34K

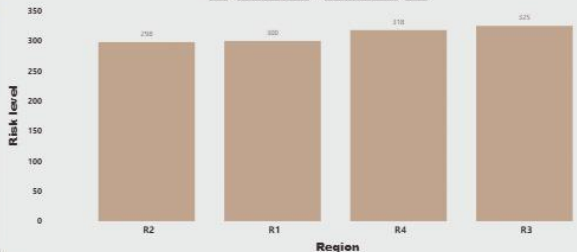
Total Forecast Error

100.41K

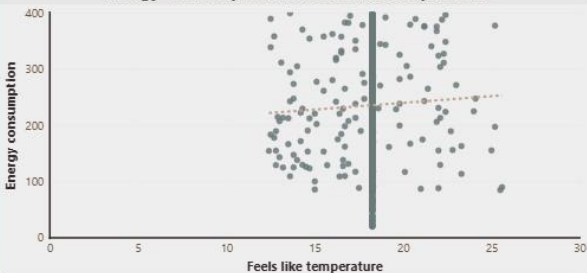
Optimization Panel: Total Consumption & Risk

region_id	Sum of energy_consumption	Sum of risk level
R3	78,193.55	77,437.06
R4	76,029.25	76,495.63
R2	74,261.20	71,523.80
R1	71,631.65	72,544.14
Total	300,115.64	298,000.63

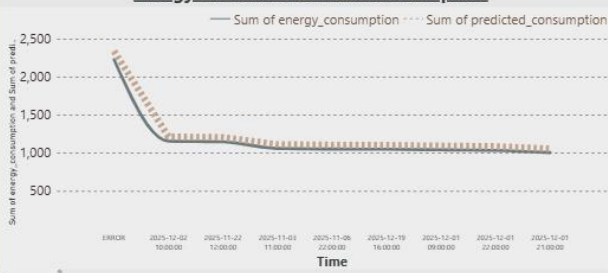
Regional Energy Risk Analysis



Energy Consumption vs. Feels Like Temperature



Energy Forecast vs. Actual Consumption



8. Conclusion

This project successfully demonstrated the potential of cloud-native data engineering and machine learning in transforming urban energy management.

1. Solving a Real-World Problem The project moved beyond theoretical analysis to address a critical challenge: grid instability and energy inefficiency. By integrating diverse datasets (Energy, Weather, Traffic), we provided a holistic view that was previously missing, enabling proactive rather than reactive management.

2. Technical Mastery of Azure Stack We successfully implemented an end-to-end Modern Data Warehouse solution. From ingesting raw data with Azure Data Factory, managing storage with Data Lake Gen2, to advanced processing with Azure Synapse Analytics. This journey solidified our understanding of cloud ETL pipelines and serverless architecture.

3. Successful Predictive Modeling The integration of Azure Machine Learning (specifically the Boosted Decision Tree model) proved that energy consumption is predictable with high accuracy. The pipeline automated the lifecycle from training to deployment, creating a reliable source of future insights.

4. Value for Smart Cities Finally, the "Optimization Engine" logic highlights the business value. By suggesting Peak Shifting and Load Balancing, this solution lays the groundwork for a true Smart City infrastructure, where data drives sustainability and cost reduction.